



جامعة محمد الأول بوجدة
UNIVERSITE MOHAMMED PREMIER OUJDA
ⵜⴰⵎⴰⵔⵜ ⵜⴰⵎⴰⵏⴰⵢⵜ ⵜⴰⵏⴰⵢⵜ ⵜⴰⵖⴰⵏⴰⵢⵜ ⵜⴰⵏⴰⵢⵜ



المدرسة الوطنية للذكاء الاصطناعي والرقمنة – بركان
ÉCOLE NATIONALE DE L'INTELLIGENCE ARTIFICIELLE ET DU DIGITAL - BERKANE
ⵎⴰⵔⵓⵏⴰⵢⵜ ⵜⴰⵏⴰⵢⵜ ⵜⴰⵖⴰⵏⴰⵢⵜ ⵜⴰⵏⴰⵢⵜ ⵜⴰⵖⴰⵏⴰⵢⵜ ⵜⴰⵏⴰⵢⵜ

Université Mohamed Premier, Oujda, Maroc
École Nationale de l'Intelligence Artificielle et du Digital, Berkane
PROJET DE FIN D'ÉTUDES
Pour l'obtention du Diplôme d'Ingénieur d'État

Filière : Intelligence Artificielle

**Nettoyage du Bruit Web par Classification
Supervisée et Embeddings Multilingues**

RÉALISÉ PAR
Yassine Ben Acha



Organisme d'accueil : Harmony Technology

Période de stage : Février à juillet 2026

Soutenu le : 05 Juillet 2026

JURY DE SOUTENANCE

Pr.	Président du jury	ENIAD
Pr.	Examineur	ENIAD
Pr. Zakaria Kaddari	Encadrant académique	ENIAD
Mme Abir Balla	Encadrante professionnelle	Harmony Technology

Remerciements

« La recherche, c'est l'art de trouver la bonne question avant même de chercher une réponse. »

Ce mémoire est le fruit de quatre mois de stage au sein de **Harmony Technology**, une expérience qui a profondément changé ma façon d'aborder le travail d'ingénieur. Avant d'en présenter les résultats, je souhaite exprimer ma reconnaissance à toutes les personnes dont le soutien et les conseils ont rendu cette expérience aussi enrichissante.

Mes premiers remerciements vont à **Mme Abir Balla**, mon encadrante professionnelle chez **Harmony Technology**, pour sa disponibilité, sa rigueur et sa capacité à recadrer le projet avec justesse quand cela était nécessaire. Ses orientations à chaque étape du travail ont été déterminantes, notamment lors des remises en question du sujet initial des moments difficiles qui se sont finalement révélés des tournants importants. Sa confiance dans ma capacité à proposer des solutions architecturales autonomes a été une vraie source de motivation.

J'adresse également mes sincères remerciements à **Pr. Zakaria Kaddari**, mon encadrant académique, pour son apport dans la définition du cadre scientifique du projet et pour ses retours toujours éclairants. Sa perspective a contribué à ancrer ce travail dans les enjeux actuels du traitement automatique du langage naturel.

Je remercie l'ensemble de l'équipe pédagogique de **l'ENIAD (École Nationale de l'Intelligence Artificielle et du Digital)**, rattachée à l'Université Mohammed Premier, pour la qualité de la formation reçue et pour avoir favorisé un environnement propice à la curiosité et à l'exigence scientifique.

Je tiens à remercier ma famille pour son soutien constant tout au long de ces quatre mois, notamment dans les moments où l'avancement du projet semblait incertain. Sa confiance m'a souvent aidé à tenir le cap mieux que n'importe quel résultat technique.

Enfin, je remercie les membres du jury d'avoir accepté d'évaluer ce travail. Je suis conscient que certains résultats restent à consolider et que plusieurs pistes méritent d'être approfondies ; leurs remarques seront, j'en suis sûr, précieuses pour la suite.

Yassine Ben Acha, 05 Juillet 2026

Résumé

Titre : Nettoyage du Bruit Web par Classification Supervisée et Embeddings Multilingues

Le web constitue aujourd’hui l’une des sources de données textuelles les plus abondantes pour les applications de traitement automatique du langage naturel (TALN). Cependant, les pages HTML collectées contiennent systématiquement une proportion importante de contenu non pertinent, communément désigné sous le terme de *bruit web* : éléments de navigation, métadonnées, contenu promotionnel, mentions légales et autres artefacts structurels qui dégradent la qualité du texte et nuisent aux traitements NLP en aval.

Ce mémoire présente **NoiseCleaner**, un pipeline modulaire de nettoyage du bruit web reposant sur une approche supervisée. Le travail s’articule autour de deux axes principaux : d’une part, la construction d’une taxonomie hiérarchique originale du bruit web, élaborée par analyse manuelle de pages web francophones et marocaines et comprenant 7 types principaux, 28 sous-types et 62 exemples réels ; d’autre part, la conception d’un jeu de données synthétique annoté fondé sur cette taxonomie, destiné à l’entraînement et à l’évaluation du système. L’architecture proposée suit un pipeline hybride en cinq étapes : segmentation du texte, filtrage heuristique par expressions régulières, génération d’embeddings sémantiques, classification supervisée des segments et reconstruction du texte nettoyé. Deux modèles d’embeddings ont été comparés : **MiniLM multilingue** et **LaBSE**, en combinaison avec deux classifieurs : la **régression logistique** et le **SVM linéaire**. Des expérimentations menées sur une version adaptée de la baseline BoilerNet (BoilerNet modifié) ont permis d’observer un gain de 8 points de F1 sur le jeu de données de référence GoogleTrends-2017. Ces résultats démontrent l’impact des optimisations apportées ; les perspectives portent sur l’enrichissement du jeu de données d’entraînement et l’approfondissement des expérimentations sur des corpus plus diversifiés.

Mots-clés

nettoyage du bruit web, traitement automatique du langage naturel, classification supervisée, embeddings multilingues, MiniLM, LaBSE, régression logistique, SVM linéaire, pipeline modulaire, jeu de données synthétique, taxonomie, BoilerNet.

Abstract

Title : Web Noise Cleaning via Supervised Text Classification and Multilingual Embeddings

The web is today one of the most abundant sources of textual data for natural language processing (NLP) applications. However, HTML pages systematically contain a significant proportion of irrelevant content, commonly referred to as *web noise* : navigation menus, metadata, promotional content, legal notices, and other structural artefacts that degrade text quality and negatively affect downstream NLP tasks.

This report presents **NoiseCleaner**, a modular web noise cleaning pipeline based on a supervised classification approach. The work is structured around two main contributions : first, the construction of an original hierarchical taxonomy of web noise, built through manual analysis of French and Moroccan web pages and comprising 7 main types, 28 subtypes, and 62 real-world examples ; second, the design of an annotated synthetic dataset derived from this taxonomy, intended for training and evaluating the system. The proposed pipeline follows a hybrid five-stage architecture : text segmentation, heuristic regex-based filtering, multilingual embedding generation, supervised segment classification, and clean text reconstruction.

Two embedding models were compared : **multilingual MiniLM** and **LaBSE**, in combination with two classifiers : **logistic regression** and **linear SVM**. Experiments conducted on an adapted version of the BoilerNet baseline (modified BoilerNet) yielded an 8-point gain in F1 score on the GoogleTrends-2017 benchmark. These results demonstrate the impact of the proposed optimizations and provide a foundation for further work, including training data enrichment and broader experimental evaluation on more varied corpora.

Keywords

web noise cleaning, natural language processing, supervised classification, multilingual embeddings, MiniLM, LaBSE, logistic regression, linear SVM, pipeline modulaire, dataset synthétique, taxonomy, BoilerNet.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des figures	ix
Liste des tableaux	x
Liste des abréviations	xi
Introduction générale	1
1 Cadre du projet et état de l'art	5
1.1 Présentation de la structure d'accueil	5
1.1.1 Harmony Technology	5
1.1.2 L'ENIAD et le cadre académique du stage	7
1.1.3 Encadrement du stage	7
1.1.4 Positionnement du projet	8
1.2 Spécificités d'un stage de recherche appliquée	8
1.2.1 Rupture avec le modèle du stage industriel classique	8
1.2.2 La gestion de l'incertitude comme compétence centrale	8
1.2.3 La collaboration avec les encadrants	9
1.3 Déroulement chronologique du stage	9
1.3.1 Vue d'ensemble en quatre phases	9
1.3.2 Phase 1 Immersion bibliographique	10
1.3.3 Phase 2 Exploration LLM	11
1.3.4 Phase 3 Modélisation du bruit	11
1.3.5 Phase 4 Conception de NoiseCleaner	11
1.4 Définition et enjeux du nettoyage du bruit web	12
1.4.1 La page web comme objet composite hétérogène	12
1.4.2 Le concept de <i>boilerplate</i> dans la littérature	12
1.4.3 Enjeux pour les applications NLP modernes	13
1.4.4 Spécificités du contexte multilingue marocain	13

1.5	État de l'art des approches de nettoyage de contenu web	14
1.5.1	Panorama général des approches	14
1.5.2	Approches à base de règles et d'heuristiques	14
1.5.3	Approches supervisées séquentielles	17
1.5.4	Approches fondées sur les grands modèles de langue	19
1.5.5	Positionnement de NoiseCleaner dans l'espace des solutions	23
1.6	Synthèse du chapter 1	24
2	Modélisation du bruit et conception de NoiseCleaner	25
2.1	Démarche de modélisation du bruit	25
2.1.1	La nécessité d'une taxonomie opérationnelle	25
2.1.2	Corpus d'analyse et protocole d'annotation	26
2.1.3	De onze types à six : les fusions justifiées	27
2.2	Taxonomie hiérarchique du bruit web	27
2.2.1	Vue d'ensemble	27
2.2.2	Sources du corpus	28
2.2.3	Type 1 Métadonnées	29
2.2.4	Type 2 Contenu Promotionnel	30
2.2.5	Type 3 UX / Interface utilisateur	32
2.2.6	Type 4 Contenu Média	33
2.2.7	Type 5 Contenu Légal	34
2.2.8	Type 6 UGC	35
2.3	De la taxonomie à l'architecture : la traitabilité différenciée comme principe de conception	37
2.3.1	La traitabilité comme critère classificatoire	37
2.3.2	Tableau de correspondance taxonomie–pipeline	37
2.4	Principes directeurs de la conception de NoiseCleaner	39
2.4.1	Supervision hybride : pragmatisme et performance	39
2.4.2	Modularité : un composant, une responsabilité	39
2.4.3	Explicabilité : tracer chaque décision	40
2.4.4	Architecture hybride supervisée : complémentarité des couches	40
2.5	Constitution du jeu de données pour l'apprentissage supervisé	40
2.5.1	Composition multilingue	41
2.6	Description détaillée des étapes du pipeline	43
2.6.1	Vue d'ensemble architecturale	44
2.6.2	Étape 1 : Segmentation	44
2.6.3	Étape 2 : Filtrage par expressions régulières	45
2.6.4	Étape 3 : Génération d'embeddings multilingues	46
2.6.5	Étape 4a : Classification supervisée (mode principal de production)	49

2.6.6	Variante non-supervisée alternative (baseline exploratoire)	52
3	Expérimentations, résultats et discussion	54
3.1	Protocole d'évaluation général	54
3.1.1	Les métriques d'évaluation en extraction de contenu	54
3.1.2	Le jeu de données de référence de BoilerNet	55
3.1.3	Les limites du jeu de données de référence pour NoiseCleaner	55
3.2	Modification de BoilerNet et gain de performance	56
3.2.1	Motivation et démarche	56
3.2.2	Les modifications apportées à BoilerNet	56
3.2.3	Résultats quantitatifs : BoilerNet original vs. BoilerNet modifié	60
3.2.4	Analyse des résultats de BoilerNet modifié	60
3.3	Évaluation de NoiseCleaner	61
3.3.1	Évaluation quantitative de la branche supervisée	62
3.3.2	Analyse qualitative de la variante exploratoire	62
3.3.3	Analyse par couche du pipeline	63
3.3.4	Analyse d'un cas de succès : Page P1	64
3.3.5	Analyse d'un cas difficile : Page P8	65
3.4	Construction du jeu de données synthétique	66
3.4.1	Motivation et objectifs	66
3.4.2	Architecture du générateur synthétique	67
3.4.3	Format des données synthétiques	67
3.4.4	Protocole d'évaluation sur données synthétiques	67
3.4.5	Résultats préliminaires sur données synthétiques	68
3.5	Discussion critique	69
3.5.1	Points forts de la démarche expérimentale	69
3.5.2	Limites identifiées et voies d'amélioration	70
3.5.3	Discussion sur les choix technologiques : UMAP et HDBSCAN dans la baseline alternative	71
3.5.4	Positionnement final de NoiseCleaner : ce qu'il est et ce qu'il n'est pas	72
4	Limites, perspectives et bilan personnel	73
4.1	Inventaire critique des limites du travail	73
4.1.1	Limites méthodologiques	73
4.1.2	Limites techniques	75
4.1.3	Limites de portée	76
4.2	Perspectives de développement	77
4.2.1	Perspectives à court terme	77
4.2.2	Perspectives à moyen terme	77
4.2.3	Perspectives à long terme	78

4.3	Bilan personnel et réflexion sur la montée en compétences	78
4.3.1	Ce que ce stage a changé dans ma façon de penser l'ingénierie . . .	79
4.3.2	Compétences techniques développées	79
4.3.3	Ce que ce stage m'a appris sur le métier de chercheur	80
	Conclusion générale	81
	Bibliographie	83
	Annexes	86
A	Glossaire technique	86
B	Format du jeu de données manuel annoté	88

Table des figures

1.1	Logo officiel de Harmony Technology	5
1.2	Organigramme fonctionnel des pôles d’expertise de Harmony Technology et positionnement du projet.	6
1.3	Photo historique regroupant la toute première promotion d’élèves-ingénieurs de l’ENIAD, entourée du corps professoral et du personnel administratif.	7
1.4	Chronologie des différentes phases du stage de fin d’études, depuis l’immersion initiale dans l’environnement de recherche jusqu’à la finalisation du pipeline NoiseCleaner et la rédaction du mémoire.	9
1.5	Architecture générale du pipeline hybride supervisé NoiseCleaner.	12
1.6	Architecture neuronale de BoilerNet basée sur une couche BiLSTM suivie d’un classificateur Sigmoid pour la détection du contenu principal et du bruit de type boilerplate.	17
1.7	Évaluation qualitative du prototype Streamlit développé pour la Phase 2. L’exemple présenté montre successivement l’identification des catégories de bruit, leur suppression par le pipeline basé sur un LLM, puis la conservation du contenu journalistique principal. Les éléments éliminés correspondent principalement aux catégories <i>Métadonnées</i> (Type 1) et <i>Contenu Média</i> (Type 4) définies dans la taxonomie du bruit.	21
2.1	Diagramme hiérarchique du Type 1 : Métadonnées	30
2.2	Diagramme hiérarchique du Type 2 : Contenu Promotionnel	31
2.3	Diagramme hiérarchique du Type 3 : UX / Interface utilisateur	32
2.4	Diagramme hiérarchique du Type 4 : Contenu Média	34
2.5	Diagramme hiérarchique du Type 5 : Contenu Légal	35
2.6	Diagramme hiérarchique du Type 6 : UGC et contenu lié	36
2.7	Distribution des classes (Content vs Noise) dans le jeu de données d’apprentissage	41
2.8	Répartition des segments par langue (Français, Arabe et Anglais)	42
2.9	Architecture générale du pipeline NoiseCleaner en modes supervisé hybride (production) et non-supervisé (exploratoire).	44

4.1	Positionnement de NoiseCleaner dans un pipeline RAG complet : il opère lors de la phase d’ingestion des documents, en amont de l’encodage et de l’indexation.	78
-----	---	----

Liste des tableaux

1.1	Déroulement chronologique du stage par phase	10
1.2	Comparaison opérationnelle Trafilatura vs. Scrapling sur le corpus de dix sites (Phase 2)	16
1.3	Analyse comparative de BoilerNet et SemText	18
1.4	Comparaison LLM local vs. LLM cloud pour le nettoyage web	20
1.5	Positionnement comparatif de NoiseCleaner	23
2.1	Récapitulatif de la taxonomie hiérarchique consolidée	28
2.8	Correspondance entre les types de la taxonomie et les couches de traitement de NoiseCleaner	38
2.9	Composition de la partition entraînement/test au niveau article (Group-ShuffleSplit)	43
2.10	Analyse comparative des modèles d’embedding candidats	47
3.1	Optimisations apportées au pipeline de traitement BoilerNet	57
3.2	Résultats comparatifs sur le jeu de données GoogleTrends-2017	60
3.3	Résultats comparatifs sur le jeu de données CleanEval	60
3.4	Performances quantitatives de la branche supervisée de NoiseCleaner sur le jeu de données MVP	62
3.5	Résultats préliminaires de NoiseCleaner par type de bruit sur le jeu de données synthétique (50 pages, évaluation préliminaire)	68

Liste des abréviations

Abréviation	Signification
TALN	Traitement Automatique du Langage Naturel
NLP	Natural Language Processing
HTML	HyperText Markup Language
LLM	Large Language Model (Grand Modèle de Langue)
BiLSTM	Bidirectional Long Short-Term Memory
CRF	Conditional Random Field
UMAP	Uniform Manifold Approximation and Projection
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
RAG	Retrieval-Augmented Generation
UGC	User-Generated Content
PFE	Projet de Fin d'Études
ENIAD	École Nationale de l'Intelligence Artificielle et du Digital
UMP	Université Mohammed Premier
MVP	Minimum Viable Product
SVM	Support Vector Machine
F1	Mesure F1 (moyenne harmonique de la précision et du rappel)

Introduction générale

Contexte général

Il existe une tension fondamentale, souvent sous-estimée, dans le traitement automatique du langage naturel appliqué au web : les données les plus abondantes sont aussi, structurellement, les moins propres. Une page d'actualité francophone typique contient, en moyenne, autant de texte parasite menus de navigation, bandeaux publicitaires, dates de publication, crédits photographiques, mentions légales, boutons de partage sur les réseaux sociaux, fils de commentaires que de contenu informatif réel. Cette réalité n'est pas un défaut marginal que l'on pourrait ignorer : elle constitue un obstacle structurel pour tout système qui consomme du texte web en entrée[1, 3].

Pour un moteur de recherche, des passages de navigation indexés au même titre que le corps de l'article dégradent la pertinence des résultats[1]. Pour un modèle de langue entraîné sur des données web brutes, les séquences de bruit injectées dans les données d'entraînement introduisent des artefacts dans les représentations apprises[10]. Pour un pipeline de *Retrieval-Augmented Generation* (RAG), des chunks extraits de menus ou de pieds de page génèrent des réponses factuellement erronées ou hors-sujet[9]. Dans tous ces cas, la qualité du nettoyage en amont conditionne directement celle du traitement en aval : aucune sophistication algorithmique appliquée à des données bruitées ne peut compenser un pré-traitement défaillant[1, 3].

Contexte du stage

Ce stage de fin d'études s'est déroulé au sein du **Département Recherche & Développement de la société Harmony Technology**, entreprise rabatée engagée dans l'intégration de solutions NTIC, IoT et RFID au service de la transformation numérique des institutions et des industriels[16, 17]. Intégrer une équipe tournée vers l'innovation, au cœur d'un environnement interdisciplinaire en prise directe avec les enjeux de numérisation du tissu économique marocain, a profondément marqué la manière dont ce projet a été abordé et conduit[17]. La mission confiée consistait à

concevoir un pipeline capable d'identifier et d'éliminer le bruit textuel présent dans le contenu extrait de pages HTML, dans un contexte multilingue (français, arabe, anglais), en combinant filtrage heuristique, représentation sémantique par embeddings et classification supervisée le tout à partir d'un jeu de données constitué et labellisé artisanalement en cours de projet[5, 8].

Ce stage revêtait une dimension pédagogique que je n'avais pas anticipée. Mes expériences précédentes s'étaient toutes inscrites dans des contextes industriels classiques : un besoin clairement formulé, un périmètre défini, une exécution balisée. Ici, rien de tout cela. Travailler au sein d'un département R&D, c'est accepter de ne pas savoir où l'on va au moment où l'on commence ; c'est apprendre à formuler des hypothèses plutôt qu'à appliquer des recettes, à remettre en question une architecture entière segmentation, filtres, choix du classifieur après des semaines de travail, et à avancer malgré l'absence de certitudes. Cette rupture de posture passer de l'ingénieur qui exécute à celui qui cherche constitue un apprentissage dont je mesure aujourd'hui la portée bien au-delà de ce seul projet.

Problématique centrale

Comment identifier et éliminer automatiquement le bruit textuel structurel, syntaxique et sémantique présent dans le contenu extrait de pages web, afin de reconstruire un texte propre et cohérent, de façon généralisable à des domaines et des langues variés, lorsque les données annotées disponibles sont rares et constituées artisanalement ?

Cette question recouvre plusieurs sous-problèmes distincts dont la résolution simultanée constitue la véritable difficulté du projet : comment définir rigoureusement ce qu'est le *bruit web* dans toute sa diversité des artefacts déterministes (balises de navigation, mentions légales, publicités) aux segments sémantiquement ambigus[1, 3] ? Comment traiter ces types de bruit aux propriétés radicalement différentes dans un pipeline unifié, alliant règles heuristiques et apprentissage supervisé[1, 2] ? Comment choisir un classifieur qui résiste au fléau de la dimensionnalité avec seulement 384 dimensions d'embedding, tout en limitant le risque de surapprentissage sur un corpus d'environ 300 segments annotés[5, 8, 14] ? Et comment concevoir une architecture suffisamment générale pour rester efficace sur des pages aussi hétérogènes que celles d'un journal marocain arabophone, d'un portail d'actualités français ou d'un site anglophone sans réentraînement spécifique à chaque domaine[8] ?

Démarche et méthodologie

La démarche adoptée pour répondre à cette problématique a suivi une progression rigoureuse en quatre phases consécutives, caractéristique d'une recherche appliquée :

1. **Phase bibliographique** : exploration systématique de l'état de l'art en extraction de contenu web, lecture approfondie des architectures existantes (Boilerpipe, Trafilatura, BoilerNet, SemText), et compréhension des enjeux théoriques et pratiques du problème[1, 3, 2].
2. **Phase expérimentale LLM** : exploration d'une approche fondée sur les grands modèles de langue, avec implémentation d'un pipeline de démonstration et évaluation comparative sur dix sites web marocains et francophones. Cette phase, bien qu'abandonnée comme solution principale, a produit des enseignements décisifs sur les limites des approches LLM en contexte de production.
3. **Phase de modélisation** : analyse manuelle du bruit sur dix pages HTML réelles, construction itérative d'une taxonomie hiérarchique en sept types principaux, et étude approfondie du code source de BoilerNet et SemText[2].
4. **Phase de conception et d'implémentation** : proposition et comparaison de deux architectures de débruitage textuel une architecture **supervisée**, retenue comme approche principale, reposant sur un pipeline en sept étapes alliant filtrage heuristique, représentation par embeddings et classification binaire ; et une architecture **non supervisée**, explorée comme alternative en l'absence de données labellisées suffisantes ; implémentation complète du pipeline *NoiseCleaner* selon l'approche supervisée, et début de la construction d'un jeu de données annoté au format JSONL destiné à l'entraînement et à l'évaluation.

Organisation du mémoire

Ce mémoire est organisé en quatre chapitres qui reflètent fidèlement la progression de la démarche de recherche.

Le **Chapitre 1** pose le cadre complet du projet : présentation de la structure d'accueil, caractérisation de la spécificité d'un stage de recherche, déroulement chronologique des travaux, analyse détaillée du problème de débruitage du contenu web, et état de l'art critique et exhaustif des approches existantes heuristiques classiques, modèles supervisés séquentiels (BoilerNet, SemText) et approches fondées sur les LLM.

Le **Chapitre 2** présente les deux contributions originales du stage : la taxonomie hiérarchique du bruit web construite empiriquement sur dix pages réelles, et l’architecture complète du pipeline *NoiseCleaner*, avec la description technique détaillée de chacune de ses sept étapes, la justification des choix technologiques (modèle d’embedding, UMAP, HDBSCAN) et l’architecture logicielle modulaire en Python.

Le **Chapitre 3** rend compte de l’expérimentation : protocole d’évaluation, résultats sur BoilerNet modifié, analyse qualitative des sorties de *NoiseCleaner*, construction du jeu de données et discussion critique sur les cas difficiles et les limites identifiées.

Le **Chapitre 4**, enfin, dresse un bilan complet du travail : limites identifiées, perspectives à court et moyen terme, et réflexion personnelle sur la montée en compétences opérée au cours de ces quatre mois.

Cadre du projet et état de l'art

1.1 Présentation de la structure d'accueil

1.1.1 Harmony Technology



Fig. 1.1 : Logo officiel de Harmony Technology

Harmony Technology est une entreprise marocaine de conseil et d'ingénierie technologique, établie à Rabat au cœur du quartier Hay Riad, où coexistent son Siège Officiel et son Centre d'Innovation. Fondée sur le manifeste *"Where Technology Meets Humanity"*, elle pose d'emblée une ambition qui dépasse le seul registre technique : celui de réconcilier la rigueur de l'ingénierie avec l'exigence de l'impact humain. Cette philosophie n'est pas un slogan de façade elle se traduit concrètement dans la culture interne de la structure, articulée autour de valeurs fondatrices dont la plus caractéristique est la *Niya*, terme arabe désignant la sincérité d'intention et l'authenticité dans l'engagement. À cette valeur cardinale s'ajoutent une exigence d'excellence sans compromis et une volonté affirmée de produire un impact mesurable et positif sur les organisations et les territoires accompagnés. C'est dans cet esprit que 110 experts ingénieurs, data scientists, architectes de solutions, spécialistes des systèmes embarqués exercent leur métier au quotidien, avec une culture de travail qui valorise autant la profondeur technique que la dimension relationnelle de chaque mission.

L'étendue de l'expertise de Harmony Technology se déploie sur quatre pôles de solutions complémentaires : le pôle **Document Intelligence & Digital Knowledge**, dédié à la transformation des contenus documentaires en connaissances exploitables ; le pôle **Smart Industry X.0**, qui couvre les domaines de l'IoT industriel et de la robotique

appliquée ; le pôle **Green Smart Cities & Territorial Intelligence**, orienté vers l'intelligence territoriale et les infrastructures urbaines durables ; et enfin, au cœur du présent projet, le pôle **AI, Data Analysis & Digital Development**, qui constitue le moteur analytique de la structure. Ce dernier pôle, opérant depuis le Centre d'Innovation de Hay Riad, concentre les compétences en apprentissage automatique, en traitement automatique des langues, en analyse de données massives et en développement de solutions digitales sur mesure. C'est précisément dans cet environnement, à l'interface entre la recherche appliquée et les besoins métier concrets des clients de l'entreprise, que s'inscrit le projet de fin d'études présenté dans ce mémoire.

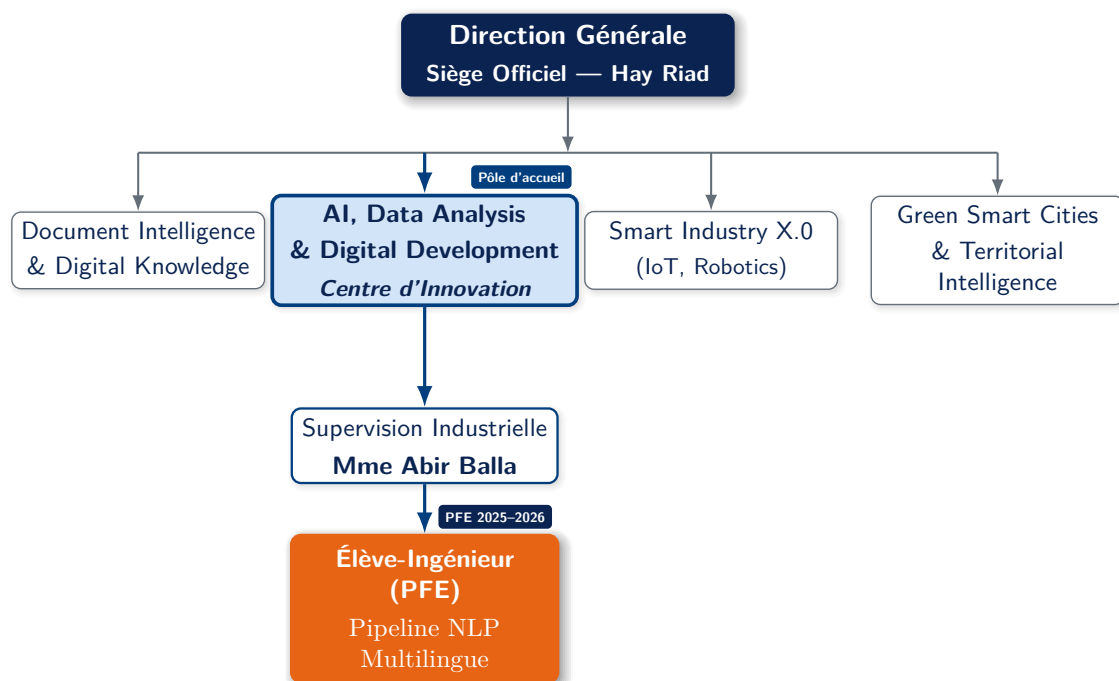


Fig. 1.2 : Organigramme fonctionnel des pôles d'expertise de Harmony Technology et positionnement du projet.

Le projet de fin d'études présenté dans ce mémoire prend racine dans un besoin métier identifié au sein du pôle **AI, Data Analysis & Digital Development** : les pipelines de traitement et d'analyse de données textuelles multilingues que le pôle déploie pour ses clients se heurtent régulièrement à la qualité dégradée des textes extraits du web : contenus parasites, bruit structurel, fragments de navigation et éléments promotionnels qui polluent les corpus avant même que tout traitement sémantique ne puisse commencer. C'est pour répondre à cette friction opérationnelle concrète que Mme **Abir Balla**, ingénieure senior au Centre d'Innovation et responsable de l'encadrement industriel de ce stage, a proposé la conception d'un pipeline NLP supervisé dédié au nettoyage automatique du bruit web multilingue. Sous sa supervision, et dans l'environnement intellectuellement stimulant du Centre d'Innovation où la proximité entre les équipes métier, les ingénieurs data et les chercheurs appliqués favorise précisément le type de démarche itérative et rigoureuse qu'exige un tel projet, ce travail a été conduit avec la

double ambition d'apporter une solution robuste à un problème industriel réel, et de contribuer modestement à la dynamique de recherche appliquée qui anime Harmony Technology au quotidien.

1.1.2 L'ENIAD et le cadre académique du stage

Ce stage s'inscrit dans le cursus de **l'École Nationale de l'Intelligence Artificielle et du Digital (ENIAD)**, composante de **l'Université Mohammed Premier d'Oujda (UMP)**, en tant que projet de fin d'études de troisième cycle d'ingénieur en Intelligence Artificielle. L'ENIAD a pour mission de former des ingénieurs spécialisés dans les disciplines de l'intelligence artificielle, de la science des données et du digital, en réponse aux besoins croissants d'un marché en pleine transformation numérique.



Fig. 1.3 : Photo historique regroupant la toute première promotion d'élèves-ingénieurs de l'ENIAD, entourée du corps professoral et du personnel administratif.

1.1.3 Encadrement du stage

L'encadrement de ce stage a été assuré conjointement par **Mme Abir Balla**, encadrante professionnelle au sein de Harmony Technology, et **Pr. Zakaria Kaddari**, encadrant académique à l'ENIAD.

Cette double supervision a offert une synergie d'expertises particulièrement bénéfique au développement du projet NoiseCleaner. D'une part, l'expertise de **Pr. Kaddari** en traitement automatique des langues (NLP) et en grands modèles de langage (LLMs)

forgée notamment par ses recherches au sein du laboratoire LaRSA a garanti la rigueur scientifique de l'approche et le bon positionnement du projet vis-à-vis de l'état de l'art.

D'autre part, la vision de **Mme Balla**, ingénieure R&D spécialisée dans l'application de l'intelligence artificielle à la cybersécurité, a été déterminante sur le plan industriel. Son exigence en matière de fiabilité numérique a permis d'orienter les choix d'ingénierie vers une architecture résiliente, capable de traiter le bruit de manière sécurisée et prête à être intégrée dans les environnements de production de l'entreprise.

1.1.4 Positionnement du projet

Le projet NoiseCleaner s'inscrit à l'intersection de plusieurs axes actifs dans la communauté NLP : la qualité des données d'entraînement pour les modèles de langue, le nettoyage de données web bruitées, et la préparation de données textuelles pour des tâches de classification supervisée. Ces enjeux convergent vers une problématique commune : comment rendre le texte web exploitable par des algorithmes d'apprentissage automatique de façon fiable et reproductible.

1.2 Spécificités d'un stage de recherche appliquée

1.2.1 Rupture avec le modèle du stage industriel classique

Avant ce stage, l'essentiel de mon expérience pratique se limitait à des environnements où le cadre du travail est clairement défini dès le départ : un cahier des charges précis, des livrables identifiés, des technologies imposées. Ce modèle, dominant dans les stages industriels classiques, laisse peu de place à l'exploration ou à la remise en question du périmètre initial.

Un stage de recherche appliquée obéit à une logique différente. Il n'existe pas, au démarrage, de solution établie au problème posé. La progression se fait par itérations : lire, comprendre, tester, réorienter. Des semaines peuvent s'écouler en phase de lecture ou d'exploration sans qu'un livrable concret soit produit. Ce n'est pas une inefficacité : c'est la condition nécessaire pour que les choix techniques ultérieurs reposent sur une compréhension réelle du domaine.

1.2.2 La gestion de l'incertitude comme compétence centrale

La compétence la plus structurante développée au cours de ce stage n'est pas exclusivement technique : c'est la capacité à travailler durablement dans l'incertitude sans la transformer en paralysie. Ce changement de posture s'est manifesté à plusieurs reprises, notamment lors du recadrage du sujet au cours du deuxième mois. Ce qui

semblait être deux semaines de travail “perdus” sur l’approche LLM s’est révélé, avec le recul, être une exploration nécessaire dont les conclusions ont directement orienté les choix architecturaux de NoiseCleaner.

1.2.3 La collaboration avec les encadrants

Un autre aspect distinctif de ce stage a été la nature du dialogue avec les encadrants. Contrairement à un contexte purement industriel où la relation est souvent de type directif, la dynamique d’encadrement ici est plus proche d’un échange entre pairs à niveaux d’expertise différents. Les encadrants posent des questions, suggèrent des pistes, remettent en cause des hypothèses, et laissent à l’étudiant la responsabilité de prendre des décisions architecturales importantes.

Cette dimension est visible dans le déroulement du projet : la suggestion de l’encadrante d’un pipeline supervisé léger (classifieur binaire sur embeddings avec un corpus annoté réduit) n’a pas été une injonction mais une proposition, que j’ai analysée et finalement retenue comme base de l’architecture supervisée de NoiseCleaner.

1.3 Déroulement chronologique du stage

1.3.1 Vue d’ensemble en quatre phases

Le stage s’est déroulé sur six mois, de février à juillet 2026, selon une progression en quatre phases dont les contours ont été partiellement redéfinis en cours de route, conformément à la logique itérative propre à la démarche de recherche appliquée.

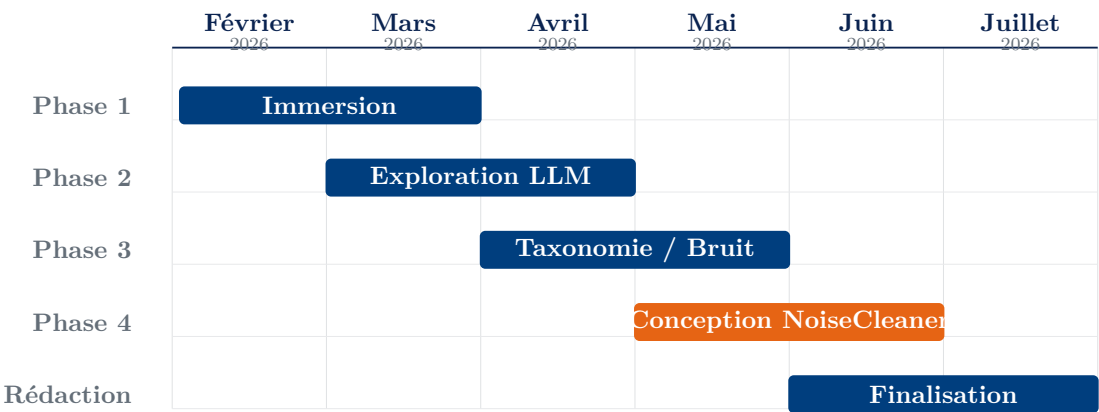


Fig. 1.4 : Chronologie des différentes phases du stage de fin d’études, depuis l’immersion initiale dans l’environnement de recherche jusqu’à la finalisation du pipeline NoiseCleaner et la rédaction du mémoire.

Tab. 1.1 : Déroulement chronologique du stage par phase

Période	Activités principales
Phase 1	Immersion bibliographique : lecture d'articles scientifiques sur le nettoyage de contenu web, étude des architectures BoilerNet et SemText, exploration des outils Trafilatura et Scraping, compréhension approfondie de la problématique générale.
Phase 2	Exploration de l'approche LLM : implémentation d'un pipeline Trafilatura + LLM de nettoyage, comparaison de modèles locaux vs. cloud, développement d'une démonstration interactive Streamlit, évaluation sur 10 sites web marocains et francophones avec fichier Excel de référence.
Phase 3	Analyse manuelle du bruit sur 10 pages HTML, construction itérative de la taxonomie hiérarchique (consolidée en 7 types principaux), lecture approfondie du code source de BoilerNet et SemText, et optimisations de BoilerNet permettant un gain de 8 points de F1 sur le benchmark GoogleTrends-2017.
Phase 4	Pivot vers l'approche supervisée : conception de l'architecture NoiseCleaner, construction du jeu de données synthétique annoté, mise en place progressive du pipeline hybride et démarrage des expérimentations comparatives.

1.3.2 Phase 1 Immersion bibliographique

Le premier mois a été entièrement consacré à la lecture et à la compréhension du domaine. Cette phase peut sembler moins productive d'un point de vue extérieur, mais elle constitue le fondement sur lequel tout le travail ultérieur a pu être construit. Sans une compréhension fine des architectures existantes leurs principes, leurs limites, leurs hypothèses implicites il aurait été difficile de proposer une approche qui apporte une valeur ajoutée réelle.

Parmi les travaux étudiés, BoilerNet et SemText ont retenu une attention particulière, non seulement au niveau des articles publiés, mais au niveau du code source lui-même. Un article scientifique présente une architecture dans ses grandes lignes ; le code source révèle les détails d'implémentation, les compromis qui ne sont jamais mentionnés dans les publications. C'est cette lecture du code qui a permis d'identifier les leviers d'amélioration exploités en phase 3.

1.3.3 Phase 2 Exploration LLM

La deuxième phase a été marquée par un premier pivot. L'objectif initial orientait naturellement la réflexion vers les LLM comme outil principal de nettoyage sémantique. Un pipeline a donc été implémenté, combinant Trafilatura pour l'extraction structurelle initiale et un LLM en mode zéro-shot pour le nettoyage fin.

Les résultats sur dix exemples de référence ont été encourageants en termes de précision brute. Cependant, une clarification du périmètre du sujet en milieu de phase a recentré le problème sur le bruit structurel et syntaxique des pages HTML complètes. Cette redéfinition a mis en évidence les limites pratiques de l'approche LLM dans ce contexte, et a ouvert la voie à l'exploration d'architectures alternatives plus légères.

1.3.4 Phase 3 Modélisation du bruit

La troisième phase constitue l'apport le plus original du stage sur le plan conceptuel. Plutôt que de chercher immédiatement à concevoir une architecture de nettoyage, la démarche adoptée a été de répondre d'abord à une question préalable fondamentale : *qu'est-ce que le bruit, précisément ?* L'analyse manuelle de dix pages HTML réelles a produit une taxonomie hiérarchique en six types principaux, dix-huit sous-types et soixante-trois exemples réels, qui constitue le fondement conceptuel de l'architecture NoiseCleaner.

Parallèlement, l'étude approfondie du code source de BoilerNet a permis d'identifier et d'implémenter plusieurs modifications ciblées sur ses composants (prétraitement, fonction de perte, régularisation et scheduler). Ces interventions ont produit un gain significatif de 8 points de F1 sur le jeu de données de référence GoogleTrends-2017, en assurant une meilleure fidélité des données et une stabilité accrue du processus d'apprentissage. Les détails techniques et le tableau récapitulatif de ces optimisations sont décrits dans le [chapter 3](#).

1.3.5 Phase 4 Conception de NoiseCleaner

La quatrième phase a été celle de la conception et de la mise en place progressive de l'architecture. La discussion avec l'encadrante autour d'une approche supervisée légère classifieur binaire entraîné sur des embeddings multilingues avec un jeu de données annoté de taille raisonnable a conduit au choix architectural final : un pipeline hybride supervisé combinant filtrage heuristique par regex, génération d'embeddings multilingues et classification supervisée.

Cette architecture a été entièrement implémentée, entraînée et évaluée sur un jeu de données annotés de 24 articles (571 segments) en français, arabe et anglais. Les résultats expérimentaux complets sont présentés au [chapter 3](#).

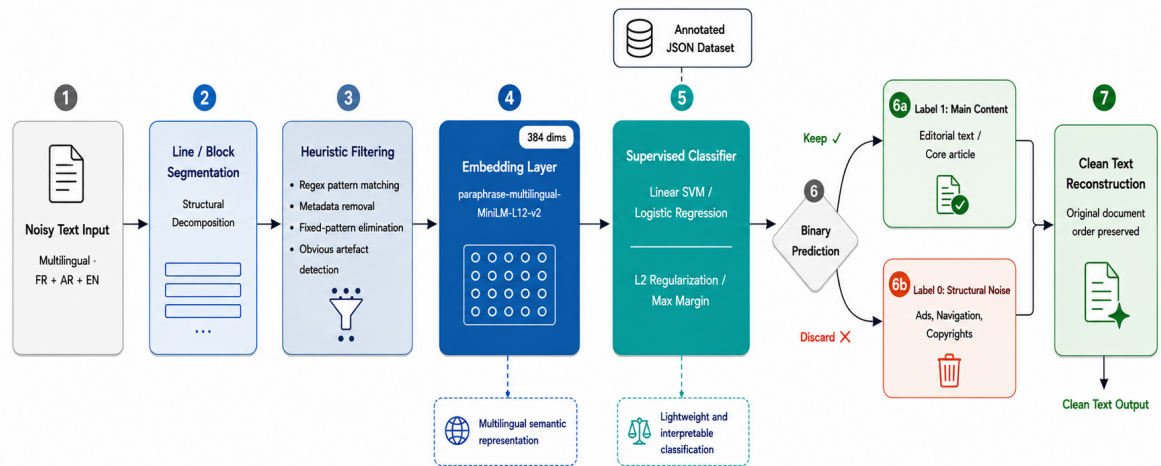


Fig. 1.5 : Architecture générale du pipeline hybride supervisé NoiseCleaner.

1.4 Définition et enjeux du nettoyage du bruit web

1.4.1 La page web comme objet composite hétérogène

Une page web moderne est, du point de vue d'un système NLP, un objet profondément hétérogène. Le HTML qui la constitue mêle dans un même flux des éléments aux fonctions sémantiques très différentes : le corps informatif de l'article, les éléments de navigation (menus, fil d'Ariane, liens internes), les métadonnées éditoriales (date, auteur, rubrique), les contenus promotionnels, les éléments d'interface utilisateur (boutons de partage, compteurs de commentaires) et les contenus légaux (mentions légales, avertissements RGPD).

Cette hétérogénéité n'est pas anecdotique : sur les dix pages du corpus de référence de ce projet, les éléments non pertinents représentent en moyenne entre 40 % et 60 % du volume textuel total brut extrait. Pour une page d'actualité francophone typique, moins de la moitié du texte visible correspond au contenu informatif que l'on cherche à conserver.

1.4.2 Le concept de *boilerplate* dans la littérature

Le terme *boilerplate* est utilisé dans la littérature NLP pour désigner l'ensemble des éléments non informatifs structurellement présents dans les pages web. Il a été popularisé dans le domaine du traitement de données web par Kohlschütter et al. (2010) [1] dans leurs travaux fondateurs sur la détection automatique du boilerplate.

La définition opérationnelle du boilerplate varie selon les travaux : certains l'en-

tendent au sens strict des éléments de navigation présents sur toutes les pages du site (header, footer, menus), d'autres l'étendent à tout élément textuel absent du flux narratif principal. Cette ambiguïté est précisément l'une des motivations qui ont conduit à la construction de la taxonomie présentée au [chapter 2](#).

1.4.3 Enjeux pour les applications NLP modernes

La qualité du nettoyage du bruit web est un enjeu transversal pour de nombreuses applications NLP :

- **Constitution de corpus d'entraînement** : les modèles de langue de grande taille sont pré-entraînés sur des corpus issus du web. La présence de bruit dans ces corpus introduit des artefacts dans les représentations apprises et peut dégrader les performances en aval [9].
- **Moteurs de recherche et indexation** : l'indexation de passages de navigation ou de mentions légales au même titre que le contenu informatif dégrade la pertinence des résultats de recherche.
- **Pipelines RAG** : dans un système de Retrieval-Augmented Generation, des passages bruités (fragments de menu, mentions de copyright) dégradent directement la qualité des réponses générées [9].
- **Veille informationnelle et agrégation** : les systèmes de veille automatisée qui résument des articles web sont directement affectés par la qualité du nettoyage initial.

1.4.4 Spécificités du contexte multilingue marocain

Ce projet présente une spécificité géolinguistique importante : les pages cibles incluent des contenus en français, en arabe et en anglais. Cette diversité linguistique introduit plusieurs défis techniques supplémentaires par rapport au cas monolingue.

Les modèles d'embedding entraînés uniquement sur des corpus anglophones ne capturent pas correctement la sémantique des segments en arabe. De même, les patterns d'expressions régulières pour la détection des métadonnées doivent être adaptés aux conventions typographiques arabes. Ces contraintes ont directement orienté le choix du modèle d'embedding `paraphrase-multilingual-MiniLM-L12-v2`, qui supporte plus de 50 langues dont l'arabe et le français, et du modèle **LaBSE** [8], conçu spécifiquement pour la représentation multilingue de phrases.

1.5 État de l'art des approches de nettoyage de contenu web

1.5.1 Panorama général des approches

La littérature sur le nettoyage de contenu web peut être organisée en trois grandes familles : les approches à base de règles et d'heuristiques, les approches supervisées par apprentissage machine, et les approches fondées sur les modèles de langue. Ces familles ne sont pas mutuellement exclusives. NoiseCleaner est lui-même une approche hybride mais leur distinction permet de structurer l'analyse critique de l'existant.

1.5.2 Approches à base de règles et d'heuristiques

Principes généraux

Les premières approches automatisées, développées dans les années 2000–2010, reposaient sur des heuristiques encodant des observations empiriques sur la structure typique des pages web : le contenu principal tend à être contenu dans des blocs textuels longs, à forte densité de texte par rapport aux balises HTML, à faible densité de liens hypertexte. Ces heuristiques sont généralement formulées sous la forme de scores par bloc, combinés dans une fonction de décision seuillée. Leur principale qualité est leur rapidité d'exécution.

Boilerpipe

Boilerpipe [1], proposé par Kohlschütter et al. en 2010, est l'un des systèmes heuristiques les plus influents dans l'histoire du nettoyage de contenu web. Il repose sur une analyse des blocs HTML, chaque bloc étant représenté par un vecteur de features simples : nombre de mots, densité de liens, position dans la page, présence de balises de formatage. Un classifieur linéaire léger, entraîné sur un corpus annoté, apprend à distinguer le contenu principal du boilerplate.

Boilerpipe se distingue par sa rapidité et son interprétabilité. Ses limites principales sont sa fragilité sur des mises en page atypiques et sa dépendance à un corpus d'entraînement potentiellement biaisé.

Trafilatura

Trafilatura [3] est un outil plus récent qui combine plusieurs stratégies heuristiques avancées : détection des zones de contenu par analyse de la densité textuelle, extraction des éléments sémantiques pertinents via des sélecteurs CSS et XPath, et nettoyage post-extraction par règles linguistiques. Il est aujourd'hui l'un des outils de référence dans la communauté pour la constitution de corpus web propres.

Lors de la phase d'exploration du stage, Trafilatura a été testé sur dix sites web marocains et francophones. Les résultats ont révélé deux types de défaillances : des cas de **sur-suppression**, où des paragraphes informatifs courts sont éliminés à tort ; et des cas de **sous-suppression**, où des blocs promotionnels structurés en paragraphes échappent au filtre.

Comparaison Trafilatura vs. Scrapling

En parallèle de Trafilatura, le framework **Scrapling** [4] a été évalué sur le même corpus de dix sites marocains et francophones constitué durant la Phase 2 du stage. Scrapling est un outil Python de scraping adaptatif de nouvelle génération, conçu explicitement pour adresser deux limitations structurelles des extracteurs classiques comme Trafilatura : l'incapacité à traiter les pages rendues dynamiquement côté client via JavaScript, et la fragilité face aux mécanismes anti-bot déployés par un nombre croissant de sites d'actualité modernes. Là où Trafilatura opère sur le HTML statique téléchargé par une requête HTTP simple, Scrapling pilote un navigateur headless (Chromium) pour récupérer le DOM final après exécution complète du JavaScript, garantissant ainsi que le contenu informatif chargé de façon asynchrone est bien présent dans le texte soumis au pipeline NLP.

Cette comparaison a mis en évidence des différences opérationnelles significatives entre les deux approches, dont les dimensions les plus pertinentes pour le contexte du projet sont résumées dans le [Table 1.2](#). Il convient de souligner que les deux outils répondent à des niveaux d'abstraction différents : Trafilatura est un extracteur de contenu (il identifie et isole le texte principal), tandis que Scrapling est avant tout un outil d'acquisition (il récupère le DOM rendu) dont les capacités d'extraction de contenu sont plus limitées. Cette distinction est fondamentale pour interpréter correctement les résultats de la comparaison.

Les résultats de cette évaluation comparative ont orienté directement les choix d'architecture de NoiseCleaner. Le constat principal est que ni Trafilatura ni Scrapling ne répondent de façon satisfaisante au besoin opérationnel central du projet : le premier dispose d'une bonne capacité d'extraction de contenu mais échoue sur les pages dynamiques (particulièrement fréquentes parmi les sites d'actualité marocains récents) ; le second permet une acquisition fiable des pages dynamiques mais ne fournit pas de module d'extraction de contenu intégré, déléguant entièrement cette responsabilité à un outil aval. Cette complémentarité partielle entre les deux outils a motivé le choix d'une architecture pipeline dans laquelle la phase d'acquisition (éventuellement assurée par Scrapling pour les sites dynamiques) est découplée de la phase de nettoyage sémantique (assurée par NoiseCleaner). C'est cette séparation des responsabilités qui garantit la modularité et l'adaptabilité du système à des contextes d'acquisition variés.

Tab. 1.2 : Comparaison opérationnelle Trafilatura vs. Scrapling sur le corpus de dix sites (Phase 2)

Critère	Trafilatura [3]	Scrapling [4]
Rendu JavaScript	Non opère sur HTML statique uniquement (requête HTTP simple)	Oui navigateur headless Chromium intégré, DOM final garanti
Gestion anti-bot	Aucune bloqué sur les sites à protection active	Native gestion des fingerprints, délais et rotation de session
Robustesse aux structures atypiques	Modérée heuristiques calibrées sur des structures HTML standard	Élevée modèle adaptatif indexant les éléments par similarité
Vitesse d'exécution	Très rapide (< 50 ms par page)	Modérée overhead navigateur (1–5 s par page selon le site)
Facilité d'intégration Python	Excellente package <code>pip</code> , API en une ligne	Bonne installation plus lourde (dépendance Playwright/Chromium)
Extraction du contenu principal	Intégrée et optimisée (algorithme dédié)	Absente nativement nécessite un post-traitement explicite
Qualité sur les sites marocains testés	Variable sur-suppression notable sur les pages à chargement asynchrone, sous-suppression sur les pages riches en contenu généré par les utilisateurs	Meilleure sur les pages dynamiques testées DOM complet récupéré, mais bruit résiduel non filtré
Maturité et maintenance	Mature projet actif, bien documenté, large adoption académique	Récent API en évolution, documentation encore incomplète

1.5.3 Approches supervisées séquentielles

Le paradigme de la classification de blocs

L'état de l'art récent est dominé par des architectures de classification de séquences, où la page HTML est découpée en blocs textuels et où chaque bloc est classifié comme *main content* ou *noise*, en tenant compte du contexte formé par les blocs adjacents. Ce paradigme représente une avancée par rapport aux approches heuristiques : au lieu de décider indépendamment pour chaque bloc, ces modèles capturent des patterns contextuels que les heuristiques locales ne peuvent pas détecter.

BoilerNet : BiLSTM avec tête Sigmoid

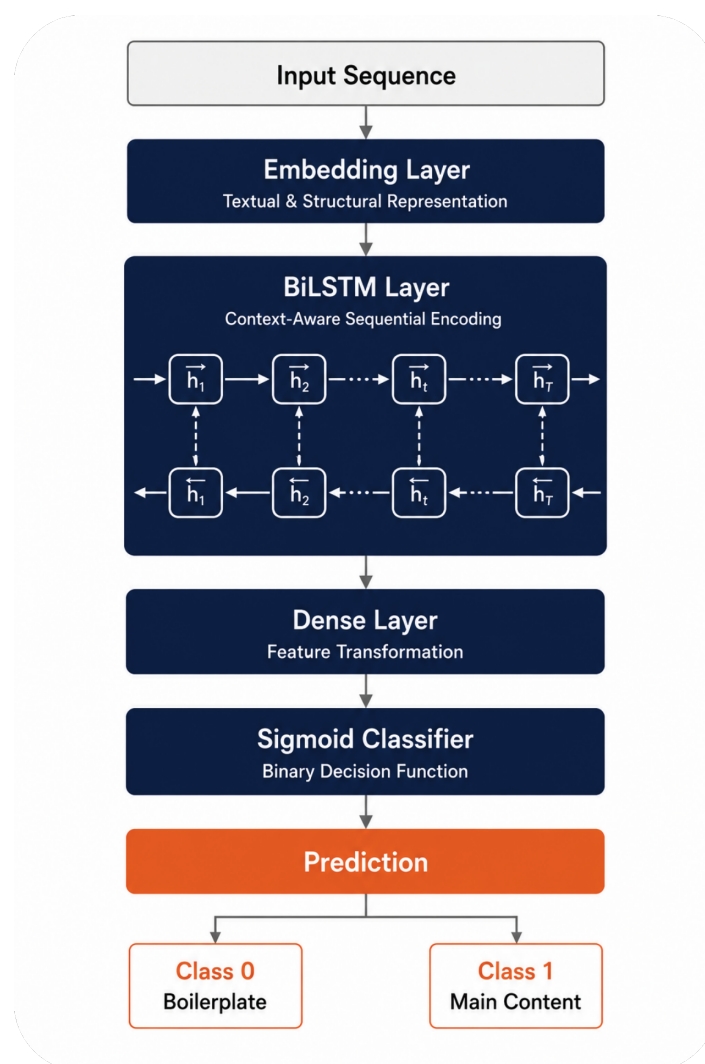


Fig. 1.6 : Architecture neuronale de BoilerNet basée sur une couche BiLSTM suivie d'un classificateur Sigmoid pour la détection du contenu principal et du bruit de type boilerplate.

BoilerNet [2], proposé par Leonhardt et al. en 2020, repose sur une architecture BiLSTM couplée à une tête de prédiction Sigmoid. La page HTML est d'abord parsée et découpée en blocs textuels selon la structure du DOM. Chaque bloc est représenté

par un vecteur de features *handcrafted* : longueur en mots, densité de liens hypertexte, position relative dans la page, présence de certaines balises de formatage. Ces vecteurs sont passés au BiLSTM, qui produit une représentation contextuelle de chaque bloc. La tête Sigmoid produit une probabilité d'appartenance au contenu principal.

L'avantage du BiLSTM est sa capacité à modéliser les dépendances à longue portée : un bloc court, sorti de son contexte, peut ressembler à du bruit, mais si ses voisins immédiats sont clairement des paragraphes informatifs, le modèle peut intégrer cette information pour le classifier correctement.

SemText : BiLSTM avec tête CRF

SemText [18] partage avec BoilerNet l'essentiel de son architecture BiLSTM, mais se distingue sur trois points fondamentaux identifiés lors de l'analyse approfondie du code source :

Tab. 1.3 : Analyse comparative de BoilerNet et SemText

Axe d'analyse	BoilerNet	SemText
Définition de l'unité de base	Bloc HTML structurel (aligné sur le DOM).	Unité sémantique, potentiellement multi-balises ; la segmentation cherche à capturer des unités de sens.
Représentation des features	Features <i>handcrafted</i> : longueur, densité de liens, position. Pas de composante sémantique explicite.	Features enrichies avec une composante sémantique partielle du contenu textuel du bloc.
Tête de prédiction	Sigmoid indépendante par bloc.	CRF (<i>Conditional Random Field</i>) : modélise les dépendances entre labels consécutifs et garantit la cohérence globale de la séquence produite. [12]

La différence la plus significative réside dans la tête de prédiction. Un Sigmoid indépendant peut produire des séquences incohérentes classifier un bloc comme *noise* alors que tous ses voisins sont *content*. Un CRF impose une contrainte de cohérence globale qui réduit ce type d'erreur.

L'analyse approfondie du code source de ces deux architectures a constitué l'une des étapes les plus formatrices du stage. Elle a mis en évidence des choix d'implémentation subtils gestion des blocs vides, normalisation des features, stratégies de padding qui ne sont jamais mentionnés dans les articles publiés. C'est cette connaissance du code qui a permis, en phase 3, d'identifier les leviers d'amélioration ayant produit un gain de 8 points de F1 sur le jeu d'évaluation de référence dans les conditions expérimentales retenues.

1.5.4 Approches fondées sur les grands modèles de langue

Le potentiel théorique des LLM pour le nettoyage

Les grands modèles de langue (LLM) représentent une alternative conceptuellement séduisante aux approches supervisées classiques. Leur capacité à comprendre le texte en contexte est, en théorie, exactement ce dont on a besoin pour distinguer un paragraphe informatif d'une mention légale ou d'un bandeau publicitaire. Un LLM utilisé en mode zéro-shot n'a besoin d'aucune donnée d'entraînement spécifique au problème, ce qui est particulièrement attractif lorsque les données annotées sont rares ou inexistantes.

Implémentation et résultats de la Phase 2

Au cours du deuxième mois du stage, un pipeline LLM a été implémenté selon l'architecture suivante :

1. **Extraction initiale** : Trafilatura effectue un premier nettoyage structurel grossier de la page HTML, en éliminant les éléments DOM manifestement non pertinents (balises `<nav>`, `<footer>`, `<header>`).
2. **Nettoyage sémantique par LLM** : le texte partiellement nettoyé est soumis à un LLM via un *prompt* soigneusement formulé, demandant au modèle de supprimer les éléments résiduels non pertinents et de retourner uniquement le texte informatif principal.
3. **Post-traitement** : une couche de règles simples est appliquée pour éliminer les artefacts propres au LLM (introductions de réponse, métacommentaires, etc.).

Une démonstration interactive a été développée sous forme d'application Streamlit, permettant d'évaluer visuellement les résultats sur dix URLs de référence chargées depuis un fichier Excel. Les évaluations ont comparé un LLM local (hébergé via Ollama) et un LLM cloud (API distante) sur les mêmes pages, comme synthétisé dans le [Table 1.4](#). Les résultats ont été encourageants en termes de précision brute sur les exemples sélectionnés. Afin d'illustrer qualitativement le comportement du pipeline, une démonstration représentative est présentée à la [Figure 1.7](#). Les annotations mettent

en évidence les catégories de bruit détectées conformément à la taxonomie élaborée durant le stage, ainsi que les informations conservées après traitement.

Tab. 1.4 : Comparaison LLM local vs. LLM cloud pour le nettoyage web

Critère	LLM Local	LLM Cloud (API)
Coût par requête	Nul (coût matériel fixe)	Variable (facturation par token)
Latence par page	10–60 secondes selon le modèle	2–10 secondes (dépendant du réseau)
Qualité du nettoyage	Bonne sur petits modèles (7B)	Excellente sur grands modèles (Qwen 32B)
Confidentialité des données	Totale	Soumise aux CGU du fournisseur
Reproductibilité	Contrôlable (température fixée)	Dépend de l'API (versions changeantes)
Disponibilité hors-ligne	Totale	Nulle

Limites structurelles de l'approche LLM

Malgré ces résultats initiaux, l'approche LLM présente des limites structurelles qui la rendent inadaptée comme solution principale dans un contexte de traitement à grande échelle.

Limites structurelles identifiées

Coût d'inférence

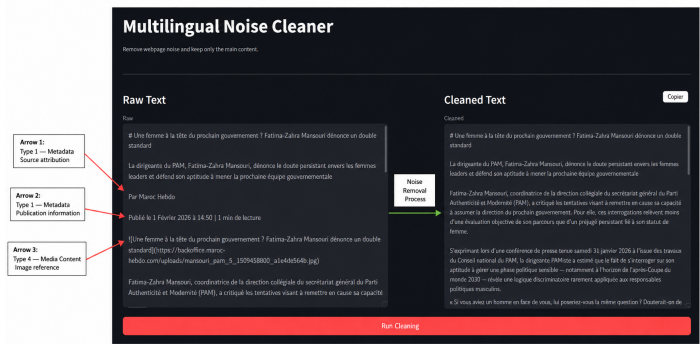
le coût de traitement d'une page via un LLM cloud est plusieurs ordres de grandeur supérieur à celui d'un classifieur léger. À l'échelle d'un corpus de milliers de pages, ce coût devient prohibitif.

Latence

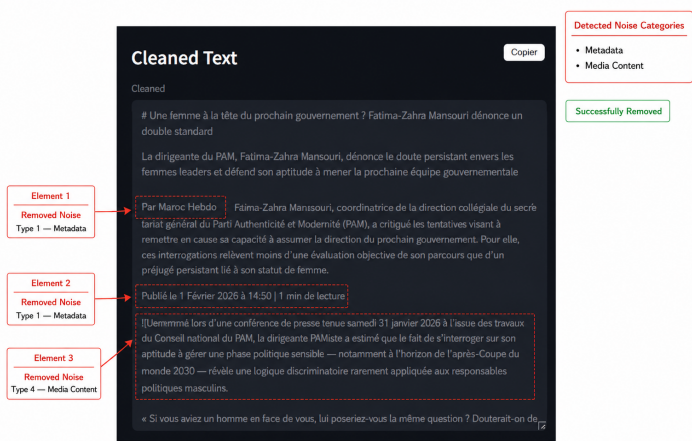
le temps de réponse, de l'ordre de plusieurs secondes par requête, est incompatible avec les exigences de débit des pipelines à grande échelle.

Dépendance à un service externe

l'utilisation d'un LLM cloud introduit des risques d'indisponibilité, de modification des conditions d'utilisation et de confidentialité des données.



(a) Identification des segments bruités dans le texte brut et comparaison avec le contenu nettoyé.



(b) Visualisation des éléments supprimés par le pipeline, principalement des métadonnées et des références média.



(c) Contenu informationnel préservé après nettoyage sémantique.

Fig. 1.7 : Évaluation qualitative du prototype Streamlit développé pour la Phase 2. L'exemple présenté montre successivement l'identification des catégories de bruit, leur suppression par le pipeline basé sur un LLM, puis la conservation du contenu journalistique principal. Les éléments éliminés correspondent principalement aux catégories *Métadonnées* (Type 1) et *Contenu Média* (Type 4) définies dans la taxonomie du bruit.

Manque de reproductibilité

les sorties des LLM varient d'une exécution à l'autre, ce qui rend l'évaluation systématique et les comparaisons entre versions difficiles.

C'est cette combinaison de contraintes, associée à la redéfinition du périmètre vers le HTML structuré, qui a conduit à écarter l'approche LLM comme solution principale. Trois enseignements déterminants en ont été tirés :

1. **La dépendance à une définition formelle du bruit** : sans taxonomie opérationnelle, l'évaluation reste subjective. Cela a motivé la phase d'annotation et de modélisation.
2. **La non-généralisabilité industrielle** : les coûts et latences disqualifient l'usage de LLM en flux tendu de production.
3. **La pertinence d'une hybridation** : l'association de règles déterministes (pour les bruits réguliers) et d'un traitement sémantique (pour les bruits ambigus) s'est avérée être l'architecture la plus robuste.

Ce pivot a ainsi permis de jeter les bases de la conception hybride supervisée de NoiseCleaner, présentée ci-après.

1.5.5 Positionnement de NoiseCleaner dans l’espace des solutions

Tab. 1.5 : Positionnement comparatif de NoiseCleaner

Critère	Heuris- tiques	BoilerNet	SemText	LLM	Noise- Cleaner
Données annotées requis	Non	Oui	Oui	Non	Oui (réduit)
Support multilingue natif	Partiel	Non	Non	Oui	Oui
Vitesse d’exécution	Très rapide	Rapide	Rapide	Lente	Modérée
Robustesse structures atypiques	Faible	Moyenne	Moyenne	Bonne	Bonne
Coût computationnel	Très faible	Faible	Faible	Très élevé	Modéré
Reproductibilité	Bonne	Bonne	Bonne	Faible	Bonne

La valeur ajoutée de l’approche hybride supervisée

NoiseCleaner combine deux couches complémentaires. La première (filtrage par expressions régulières) hérite de la simplicité et de l’interprétabilité des approches heuristiques, et l’applique précisément aux types de bruit pour lesquels ces qualités sont suffisantes : les métadonnées de publication, les attributions de source et les mentions légales suivent des patterns syntaxiques stables, détectables de façon déterministe.

La couche suivante (embeddings multilingues + classifieur supervisé) apporte la richesse des représentations sémantiques modernes pour traiter les types de bruit sémantiquement ambigus, tout en restant reproductible et applicable à grande échelle. C’est cette complémentarité, directement motivée par la taxonomie du bruit présentée au [chapter 2](#), qui constitue la contribution architecturale centrale de NoiseCleaner.

1.6 Synthèse du chapter 1

Ce premier chapitre a posé les fondations du projet NoiseCleaner à trois niveaux complémentaires.

Au niveau institutionnel, il a présenté le cadre du stage : Harmony Technology comme structure d'accueil professionnelle, l'ENIAD comme établissement de formation académique, et la spécificité d'une démarche de recherche appliquée par rapport à un stage de développement classique.

Au niveau problématique, il a défini le problème adressé : le nettoyage automatique du bruit web dans des pages HTML, dans un contexte multilingue, en vue d'améliorer la qualité des données textuelles pour des tâches NLP en aval.

Au niveau de l'état de l'art, il a dressé un panorama critique des approches existantes : heuristiques (Boilerpipe, Trafilatura), supervisées séquentielles (BoilerNet, SemText) et fondées sur les LLM. Ce panorama constitue le fondement sur lequel les choix architecturaux de NoiseCleaner, présentés au [chapter 2](#), trouvent leur justification.

Modélisation du bruit et conception de NoiseCleaner

La conception d'un système de nettoyage automatique du bruit web ne peut pas commencer par le choix d'un algorithme ou d'un modèle d'apprentissage automatique. Elle doit commencer par une question préliminaire plus fondamentale, trop souvent négligée dans la littérature : *qu'est-ce que le bruit, précisément ?* Sans réponse rigoureuse à cette question, tout système de nettoyage opère sur une définition implicite, non contrôlée et non évaluable du problème qu'il cherche à résoudre.

Ce chapitre présente les deux contributions originales majeures de ce stage. La première est une taxonomie hiérarchique du bruit web, construite empiriquement par analyse manuelle de dix pages réelles, qui fournit une définition opérationnelle et multi-niveaux de ce qu'il convient d'éliminer. La seconde est l'architecture complète du pipeline NoiseCleaner, dont chaque décision de conception est directement motivée par les propriétés identifiées dans la taxonomie.

Ces deux contributions sont intimement liées et ne peuvent être comprises indépendamment l'une de l'autre : la taxonomie n'est pas un exercice théorique déconnecté du système elle en est le fondement architectural.

2.1 Démarche de modélisation du bruit

2.1.1 La nécessité d'une taxonomie opérationnelle

La revue de la littérature présentée au [chapter 1](#) a mis en évidence une lacune récurrente dans les approches existantes : la définition du bruit y est invariablement binaire et implicite. BoilerNet [2] classe chaque bloc en *main content* ou *boilerplate* ; SemText [18] fait de même avec une modélisation séquentielle plus riche ; les approches heuristiques appliquent des seuils sur des features structurelles sans jamais formaliser ce que le bruit englobe réellement. Cette dichotomie binaire est fonctionnellement simple,

mais elle masque une réalité beaucoup plus hétérogène : le “bruit” d’une page web n’est pas une entité homogène. Il recouvre des objets textuels aux natures radicalement différentes, qui n’ont ni les mêmes propriétés linguistiques, ni les mêmes propriétés structurelles, ni les mêmes degrés de traitabilité par des approches algorithmiques.

Un bouton de partage sur les réseaux sociaux (“Partager sur Facebook”) n’a rien en commun, du point de vue sémantique, avec une mention de copyright (“© 2026 Tous droits réservés”), avec une légende de photo (“Vue aérienne du centre de Rabat, 2024”), ou avec un appel à l’abonnement (“Accédez à tous nos articles sans publicité pour 5,99 € / mois”). Ces quatre éléments sont tous du bruit dans le sens où aucun ne fait partie du flux narratif principal de l’article mais ils diffèrent profondément dans leur structure, leur longueur, leur variabilité lexicale et leur traitabilité algorithmique. Une taxonomie qui les distingue permet de leur appliquer des stratégies de filtrage adaptées ; une dichotomie binaire les traite indistinctement et perd une information précieuse.

2.1.2 Corpus d’analyse et protocole d’annotation

La taxonomie a été construite à partir de l’analyse manuelle de dix pages web francophones et marocaines, sélectionnées pour leur diversité structurelle et éditoriale. Le corpus couvre plusieurs types de sites d’actualité : portails d’information généraliste, journaux en ligne, sites d’actualité régionale marocaine, et agrégateurs de contenus. Cette diversité délibérée était essentielle pour garantir que la taxonomie capture des types de bruit effectivement présents dans des contextes variés, et non pas les artefacts spécifiques d’un seul type de site.

Le protocole d’annotation suivi était le suivant :

1. **Extraction brute** : chaque page est téléchargée et son texte brut est extrait sans aucun pré-traitement, en conservant tous les segments textuels visibles, y compris les éléments d’interface.
2. **Annotation segment par segment** : chaque segment textuel est examiné individuellement et labellisé : soit *content* (appartient au flux narratif principal de l’article), soit rattaché à l’un des types de bruit de la taxonomie en cours de construction.
3. **Identification et nommage des types** : lorsqu’un segment ne correspond à aucun type déjà identifié, un nouveau type est créé avec une définition provisoire.
4. **Révision et consolidation itérative** : après l’annotation de chaque page, la taxonomie est révisée : les types redondants sont fusionnés, les définitions sont affinées, et les exemples sont réattribués si nécessaire.

Ce processus a produit, à l’issue de l’annotation de la première page, une liste initiale de types bruts. Cette liste s’est enrichie et transformée au fil des pages, pour atteindre

onze types à l'issue de l'analyse complète du corpus. Une phase de consolidation finale a réduit ce nombre à six par fusion des types présentant des redondances sémantiques, sans supprimer aucun exemple.

2.1.3 De onze types à six : les fusions justifiées

La réduction de onze à six types mérite d'être documentée explicitement, car elle illustre la nature itérative et rigoureuse de la démarche de modélisation. Les fusions réalisées n'ont pas été arbitraires : chacune a été motivée par une redondance sémantique réelle entre les types concernés, vérifiable sur les exemples affectés.

La règle appliquée était la suivante : deux types peuvent être fusionnés si et seulement si la distinction entre eux ne modifie pas la stratégie de filtrage à appliquer c'est-à-dire si les mêmes outils algorithmiques peuvent les traiter de façon identique. Cette règle garantit que la taxonomie consolidée est non seulement plus parcimonieuse, mais aussi directement exploitable pour la conception du pipeline.

Nota bene : conformément à cette règle, aucun exemple réel n'a été supprimé lors des fusions ; chaque exemple a été réattribué au type consolidé correspondant. Le volume total de 63 exemples est donc intégralement conservé dans la version finale.

2.2 Taxonomie hiérarchique du bruit web

2.2.1 Vue d'ensemble

La taxonomie finale comprend six types principaux, consolidés par fusions raisonnées pour éliminer les redondances sémantiques et structurelles, s'appuyant sur soixante-trois exemples réels extraits du corpus.

Tab. 2.1 : Récapitulatif de la taxonomie hiérarchique consolidée

Type principal	Sous-types correspondants	Nombre d'exemples
Type 1 : Métadonnées	Date, Heure, Source, Auteur, Titre dupliqué, Temps de lecture	14
Type 2 : Contenu Promotionnel	Newsletters, Appel à action (CTA), Avantages	25
Type 3 : UX / Interface utilisateur	Tracking, Commentaires, Chargement	3
Type 4 : Contenu Média	Images, Crédits, Légendes	7
Type 5 : Contenu Légal	Copyright, Mentions légales, Restrictions, Paywall	4
Type 6 : UGC	Navigation, Contenu lié	10
TOTAL		63

Ce que ce tableau ne montre pas immédiatement, c'est la disparité profonde entre les types en termes de variabilité sémantique et de difficulté de filtrage. Le Type 1 est le plus simple : ses exemples suivent des patterns syntaxiques très stables ("Publié le [date]", "Mis à jour le [date]") et sont traçables par des expressions régulières simples. Le Type 2 (Contenu Promotionnel), au contraire, est le plus hétérogène et le plus difficile : avec vingt-cinq exemples couvrant des publicités, des appels à l'abonnement, des newsletters et des suggestions d'articles connexes, sa variabilité sémantique est telle que seule une approche fondée sur la représentation sémantique profonde peut l'adresser de façon robuste.

2.2.2 Sources du corpus

Le corpus analysé est composé de 10 textes extraits de pages web d'actualités marocaines et francophones. Ces sources ont été sélectionnées pour représenter la

diversité des structures éditoriales et des pratiques de développement web :

- **Texte 1** — Nomination Fatima-Zahra Mansouri (Maroc Hebdo) : [Consulter](#)
- **Texte 2** — Article d’actualité générale (Rue20) : [Consulter](#)
- **Texte 3** — Investissements et banque d’affaires (TelQuel) : [Consulter](#)
- **Texte 4** — Match Maroc-Sénégal au stade Moulay Abdellah (Le Desk) : [Consulter](#)
- **Texte 5** — Actualité politique - PAM (Hespress) : [Consulter](#)
- **Texte 6** — Réseaux sociaux et CAN 2025 (Le360) : [Consulter](#)
- **Texte 7** — Technologie batteries solides (Numerama) : [Consulter](#)
- **Texte 8** — Actualité générale (Anahar News) : [Consulter](#)
- **Texte 9** — Détecteur de rumeurs : Blue Monday (Science Presse) : [Consulter](#)
- **Texte 10** — Article d’information (Hibazoom) : [Consulter](#)

2.2.3 Type 1 Métadonnées

Définition

Une métadonnée est tout attribut qui fournit une information sur le contenu principal (*main content*) sans en faire partie directement. Elle comprend les dates de publication et de mise à jour, l’heure, la source éditoriale originelle, le nom de l’auteur, les mentions de localisation géographique, les titres dupliqués, le temps estimé de lecture et les notes éditoriales d’actualisation.

Propriétés et traitabilité

La caractéristique principale de ce type est sa régularité syntaxique et structurelle. D’une part, les métadonnées temporelles impliquent presque toujours la présence d’une date (sous la forme d’une combinaison de chiffres et de noms de mois) ou d’un indicateur temporel relatif. Cette propriété les rend directement accessibles au filtrage par expressions régulières, sans nécessiter de compréhension sémantique profonde.

D’autre part, les attributions de source présentent une variabilité de formulation plus importante, mais restent largement traçables par expressions régulières grâce à des introducteurs stables (“Par :”, “Écrit par”, “Rédaction”, “AFP”, “Reuters”). La principale difficulté réside dans la distinction entre une attribution de source et une citation dans le corps de l’article (ex : “Selon Jean Dupont...” vs. “Par Jean Dupont”). Cette ambiguïté peut être levée en exploitant des règles de positionnement géométrique ou séquentiel, ces éléments apparaissant presque exclusivement en début ou en fin de document.

Exemples caractéristiques

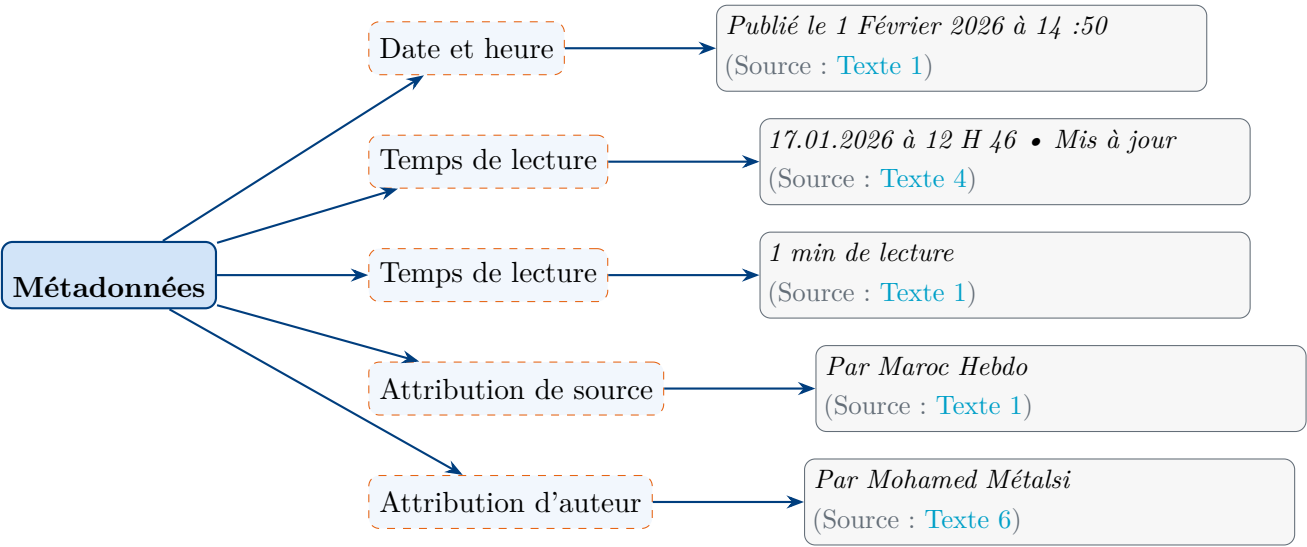


Fig. 2.1 : Diagramme hiérarchique du Type 1 : Métadonnées

Sous-type	Exemple extrait
Date et heure n	Publié le 1 Février 2026 à 14 :50 (Source : Texte 1)
Temps de lecture	منذ يومين (Source : Texte 2)
Attribution de source	Par Maroc Hebdo (Source : Texte 1)
Attribution d'auteur	Par Mohamed Métalsi (Source : Texte 6)

2.2.4 Type 2 Contenu Promotionnel

Définition

Le contenu promotionnel regroupe l'ensemble des segments textuels à visée marketing, commerciale ou incitative présents sur une page web. Ces éléments sont considérés comme du bruit sémantique car ils ne contribuent pas à la substance informative ou journalistique de l'article (ex : incitations à s'abonner, newsletters, boutons d'offres exclusives et avantages premium).

Propriétés et traitabilité

Ce type est de loin le plus difficile à filtrer de façon automatique, pour plusieurs raisons cumulatives :

- **Variabilité lexicale extrême** : le vocabulaire des contenus promotionnels est délibérément varié et renouvelé, conçu pour échapper aux filtres publicitaires et pour

engager l'attention du lecteur. Il n'existe pas de liste fermée de termes ou de patterns syntaxiques permettant de les identifier exhaustivement.

- **Structure de phrase normale** : contrairement aux éléments UX (très courts et formulaires), les contenus promotionnels peuvent être rédigés sous la forme de phrases grammaticalement complètes, voire de paragraphes entiers, avec une structure syntaxique indistinguishable du contenu informatif.
- **Contextualisation éditoriale** : les publicités natives et les partenariaux éditoriaux sont délibérément conçus pour ressembler au contenu éditorial. Ils utilisent le même style rédactionnel, la même typographie et parfois les mêmes structures narratives, rendant leur identification difficile même pour un lecteur humain.
- **Positionnement variable** : à la différence des métadonnées (généralement en début d'article) ou des crédits (en fin d'article), les contenus promotionnels peuvent apparaître n'importe où dans la page : avant l'article, après, au milieu, dans des encadrés insérés entre les paragraphes.

Ces caractéristiques font du Contenu Promotionnel le test principal de robustesse de NoiseCleaner. C'est précisément pour adresser ce type de bruit que la couche sémantique de classification supervisée (Phase 2) est indispensable : seule une représentation sémantique profonde permet de distinguer un appel à l'abonnement inséré entre deux paragraphes d'un article informatif.

Exemples caractéristiques

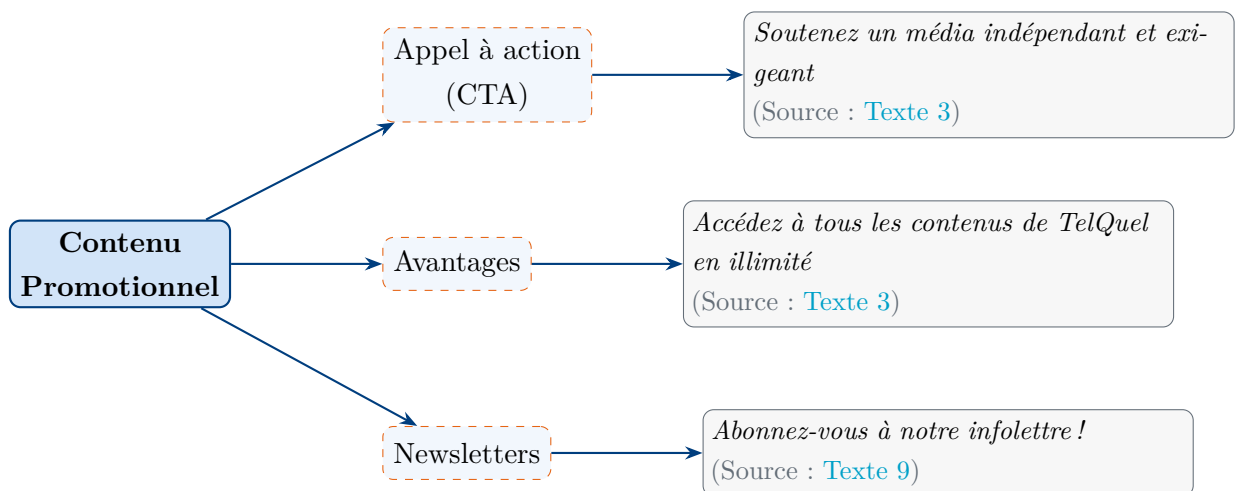


Fig. 2.2 : Diagramme hiérarchique du Type 2 : Contenu Promotionnel

Sous-type	Exemple extrait
Appel à action (CTA)	<i>Soutenez un média indépendant et exigeant</i> (Source : Texte 3)
Avantages	<i>Accédez à tous les contenus de TelQuel en illimité</i> (Source : Texte 3)
Newsletters	اشترك الآن في النشرة البريدية لجريدة هسبريس، لتصلك آخر الأخبار يوميا (Source : Texte 5)
Appel à action (CTA)	<i>Découvrez les nombreux avantages de Numerama+.</i> (Source : Texte 7)

2.2.5 Type 3 UX / Interface utilisateur

Définition

Ce type regroupe les éléments d'interface utilisateur destinés à l'interaction : indicateurs de lecture, messages de suivi d'activité, éléments liés à l'ajout ou au chargement dynamique de commentaires, boutons de partage social et compteurs d'interaction.

Propriétés et traitabilité

Les éléments UX sont parmi les plus faciles à identifier syntaxiquement : ils ont tendance à être très courts (un à cinq mots), à contenir des chiffres combinés à des unités ("3 min de lecture", "2,4K vues", "15 commentaires"), ou à suivre des patterns d'invitation à l'action très stables ("Partager sur", "S'abonner à", "Voir les commentaires"). La liste des patterns caractéristiques est fermée et stable ; elle peut être constituée exhaustivement sur le corpus de référence et appliquée par expressions régulières.

Exemples caractéristiques

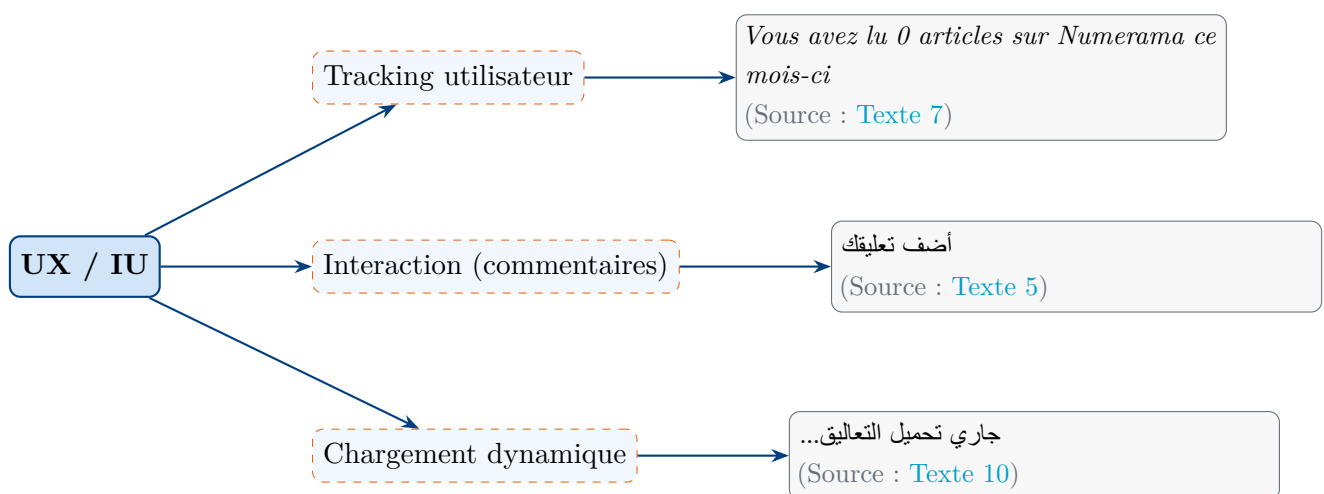


Fig. 2.3 : Diagramme hiérarchique du Type 3 : UX / Interface utilisateur

Sous-type	Exemple extrait
Tracking	<i>Vous avez lu 0 articles sur Numerama ce mois-ci</i> (Source : Texte 7)
Commentaires	أضف تعليقك (Source : Texte 5)
Chargement	جاري تحميل التعليق... (Source : Texte 10)

2.2.6 Type 4 Contenu Média

Définition

Le Contenu Média regroupe les textes directement associés aux éléments multimédias intégrés dans la page : légendes de photographies, crédits photographiques, descriptions de vidéos, transcriptions ou résumés de podcasts intégrés, ainsi que les liens URL absolus de ces ressources.

Propriétés et traitabilité

C'est précisément cette ambiguïté qui rend ce type difficile à traiter par des règles simples. Un filtre regex peut identifier les crédits photographiques ("Photo : Agence Reuters", "Crédit : Ahmed Benali") qui suivent des patterns stables ; mais les légendes elles-mêmes sont des phrases grammaticalement complètes, thématiquement liées à l'article, et syntaxiquement indistinguables du contenu principal.

La stratégie retenue dans NoiseCleaner est de laisser les éléments de Contenu Média passer le filtre Regex Phase 1 (sauf les crédits photographiques explicitement identifiés) et de les soumettre à la couche de classification supervisée (Phase 2) en production.

Exemples caractéristiques

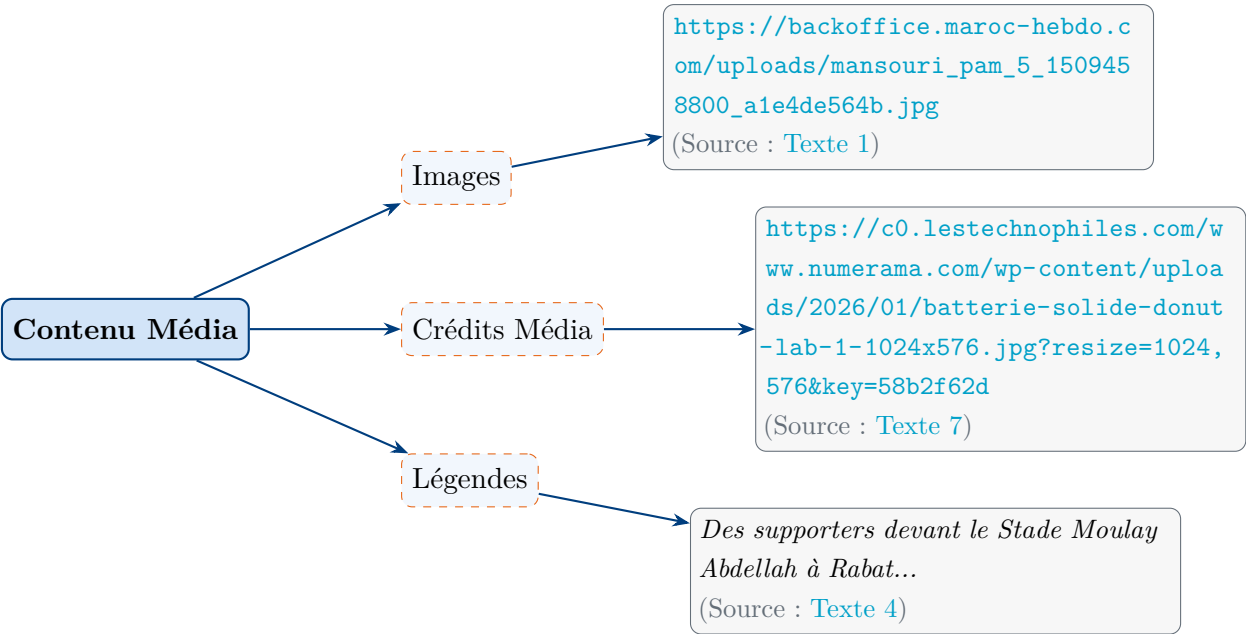


Fig. 2.4 : Diagramme hiérarchique du Type 4 : Contenu Média

Sous-type	Exemple extrait
Images	https://backoffice.maroc-hebdo.com/uploads/mansouri_pam_5_1509458800_a1e4de564b.jpg (Source : Texte 1)
Légendes	Des supporters devant le Stade Moulay Abdellah à Rabat, avant un match de la CAN 2025. Crédit : Le Desk (Source : Texte 4)

2.2.7 Type 5 Contenu Légal

Définition

Ce type regroupe les éléments juridiques présents dans les pages web : restrictions d’accès de type paywall, avis de droits d’auteur (copyright), mentions légales et avertissements de modération des commentaires.

Propriétés et traitabilité

La principale qualité de ce type du point de vue du filtrage est sa régularité syntaxique et sémantique. Les mentions de copyright (“© 2026 Tous droits réservés”, “Reproduction interdite sans autorisation”) suivent des patterns stables et directement traçables par expressions régulières. Les avertissements RGPD sont souvent longs, mais

leur vocabulaire est suffisamment spécialisé (“données personnelles”, “cookies”, “tiers de confiance”, “consentement”) pour permettre une détection fiable.

Ce type est entièrement traité par la couche Regex Phase 1 de NoiseCleaner ; aucun élément de Contenu Légal n’est transmis à la couche de classification sémantique.

Exemples caractéristiques

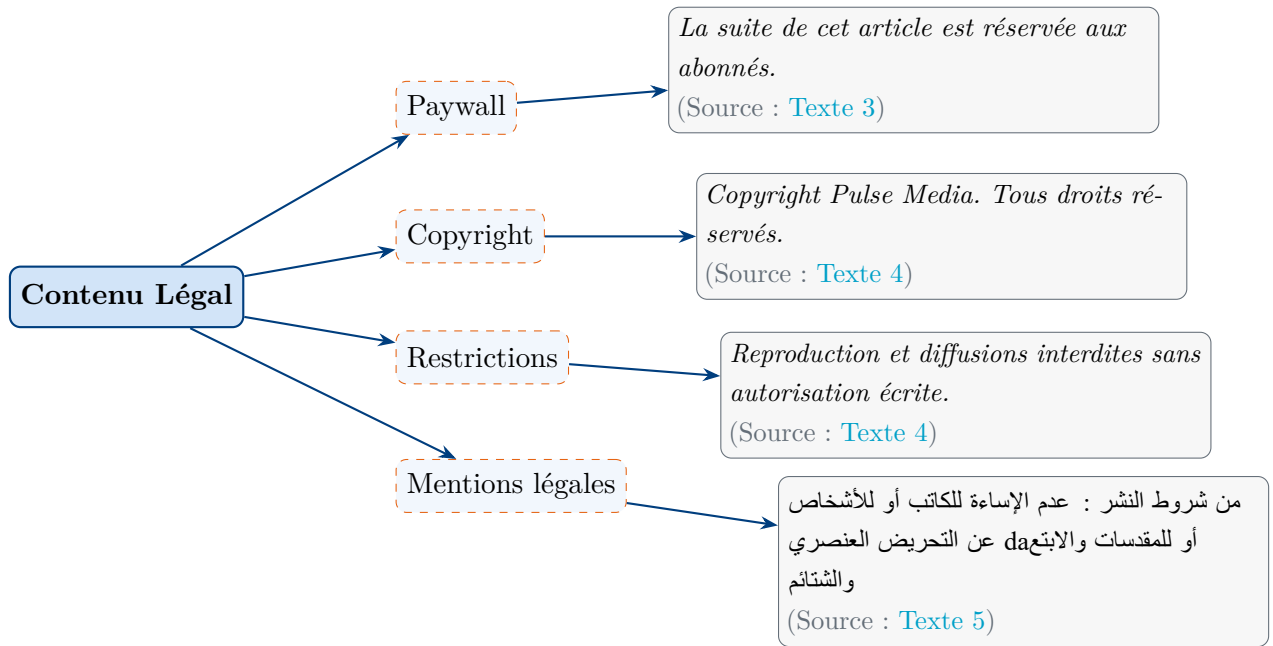


Fig. 2.5 : Diagramme hiérarchique du Type 5 : Contenu Légal

Sous-type	Exemple extrait
Paywall	<i>La suite de cet article est réservée aux abonnés.</i> (Source : Texte 3)
Copyright	<i>Copyright Pulse Media. Tous droits réservés.</i> (Source : Texte 4)
Restrictions	<i>Reproduction et diffusions interdites sans autorisation écrite.</i> (Source : Texte 4)
Mentions légales	<i>من شروط النشر : عدم الإساءة للكاتب أو للأشخاص أو للمقدسات والابتعاد عن التحريض العنصري والشتائم</i> (Source : Texte 5)

2.2.8 Type 6 UGC

Définition

Ce type regroupe les éléments de navigation interne (suggestions de lecture ou articles liés), le contenu généré par les utilisateurs (*User Generated Content UGC*) comme les commentaires ou évaluations, ainsi que les notes d'architecture éditoriale.

Propriétés et traitabilité

Ce type présente une traitabilité mixte. D'une part, les éléments de navigation interne (ex : menus, liens de fil d'Ariane) ou les titres de sections fonctionnelles sont courts, formulaires et sémantiquement stables, ce qui permet de les intercepter facilement via des filtres par expressions régulières (Regex Phase 1). D'autre part, le contenu généré par les utilisateurs (UGC) comme les commentaires et les suggestions d'articles liés peuvent être rédigés en langue naturelle et sémantiquement proches du sujet de l'article principal. Pour ces derniers, les filtres déterministes s'avèrent inefficaces. La stratégie dans NoiseCleaner consiste à confier l'élimination des commentaires volumineux et des recommandations d'articles à la couche de classification supervisée (Phase 2) de production, qui exploite les embeddings sémantiques pour isoler le bruit du contenu principal.

Exemples caractéristiques

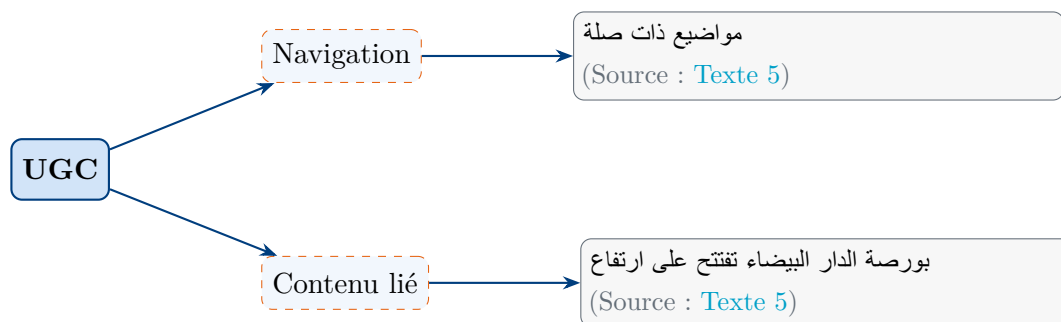


Fig. 2.6 : Diagramme hiérarchique du Type 6 : UGC et contenu lié

Sous-type	Exemple extrait
Navigation	مواضيع ذات صلة (Source : Texte 5)
Navigation	À lire également (Source : Texte 9)
Contenu lié	بورصة الدار البيضاء تفتتح على ارتفاع rus (Source : Texte 5)

2.3 De la taxonomie à l'architecture : la traitabilité différenciée comme principe de conception

2.3.1 La traitabilité comme critère classificatoire

Au-delà de sa valeur descriptive, la taxonomie construite présente une propriété architecturale fondamentale : elle permet de partitionner les types de bruit en deux grandes classes selon leur **traitabilité algorithmique**, c'est-à-dire selon la nature des outils nécessaires pour les identifier et les supprimer de façon fiable.

La première classe, que l'on appellera **bruit déterministe**, regroupe les types de bruit dont l'identification ne requiert pas de compréhension sémantique du contenu : leurs propriétés syntaxiques et structurelles sont suffisamment stables et distinctives pour être capturées par des règles formelles. Elle comprend les Types 1 (Métadonnées), 3 (UX/Interface, partiellement), et 5 (Contenu Légal).

La seconde classe, le **bruit sémantiquement ambigu**, regroupe les types dont l'identification requiert une représentation du sens du contenu, car leur forme syntaxique ne suffit pas à les distinguer du contenu principal. Elle comprend les Types 2 (Contenu Promotionnel), 4 (Contenu Média) et 6 (UGC, partiellement).

2.3.2 Tableau de correspondance taxonomie–pipeline

Ce tableau est la pièce maîtresse de la justification architecturale de NoiseCleaner : il établit le lien direct entre chaque type de la taxonomie et la couche du pipeline supervisé responsable de son traitement.

Tab. 2.8 : Correspondance entre les types de la taxonomie et les couches de traitement de NoiseCleaner

Type	Classe	Couche de traitement
Type 1 Métadonnées	Déterministe	Phase 1 Regex (<i>patterns stables : dates, horodatages, attributions de source, indicateurs temporels relatifs</i>)
Type 2 Contenu Promotionnel	Ambigu	Phase 2 Classifieur supervisé (<i>MiniLM + régression logistique ; newsletters, CTA, avantages abonnement</i>)
Type 3 UX / Interface	Semi-déterministe	Phase 1 Regex pour les patterns stables (<i>tracking, boutons d'action</i>); Phase 2 Classifieur supervisé pour les cas ambigus
Type 4 Contenu Média	Ambigu	Phase 1 Regex pour les crédits photographiques (<i>patterns fixes</i>); Phase 2 Classifieur supervisé pour les légendes (<i>phrases complètes sémantiquement ambiguës</i>)
Type 5 Contenu Légal	Déterministe	Phase 1 Regex (<i>patterns fixes : copyright, paywall, mentions légales, restrictions de diffusion</i>)
Type 6 Structure / UGC / Éditorial	Mixte	Phase 1 Regex pour la navigation et les tags structurels; Phase 2 Classifieur supervisé pour l'UGC et les encadrés éditoriaux

Ce tableau démontre que l'architecture en deux couches de NoiseCleaner n'est pas un choix arbitraire : elle est la conséquence directe et nécessaire de la structure de la taxonomie. Toute architecture qui ne distinguerait pas ces deux classes de traitabilité serait soit inefficace (appliquer un clustering sémantique coûteux à des métadonnées de publication trivialement détectables), soit insuffisante (appliquer des règles syntaxiques

au contenu promotionnel ambigu).

2.4 Principes directeurs de la conception de NoiseCleaner

2.4.1 Supervision hybride : pragmatisme et performance

NoiseCleaner repose en production sur une architecture supervisée hybride. Bien que des approches non supervisées basées sur le clustering aient été explorées à des fins comparatives lors des phases préliminaires, les exigences opérationnelles (déterminisme, performance et latence) ont imposé le choix d'un apprentissage supervisé.

Cette architecture en production combine deux phases complémentaires : un filtrage déterministe par expressions régulières (Phase 1) pour éliminer les bruits structurels, suivi d'une classification supervisée légère (Phase 2) sur des représentations vectorielles denses (plongements de phrases). L'annotation manuelle d'un échantillon de segments issus de 24 articles de presse a permis d'entraîner des classifieurs robustes (SVM linéaire [21] ou régression logistique). Cette approche concilie l'efficacité sémantique et la rapidité d'exécution requises en production, tout en s'affranchissant de l'instabilité inhérente au clustering non supervisé.

2.4.2 Modularité : un composant, une responsabilité

La modularité est le second principe architectural central de NoiseCleaner. Elle découle d'une expérience empirique bien documentée en génie logiciel : les systèmes dont chaque composant a une responsabilité unique et bien définie sont plus faciles à tester, à déboguer, à faire évoluer et à réutiliser.

Dans NoiseCleaner, ce principe se traduit par une architecture où chaque étape du pipeline correspond à un module Python indépendant, avec une interface d'entrée/sortie clairement typée. On peut remplacer le modèle d'embedding (Étape 3) sans toucher au code des autres étapes. On peut désactiver la réduction de dimension (Étape 4) via un paramètre de configuration sans modifier le reste du pipeline. On peut substituer le classifieur supervisé par un autre modèle (Étape 4a) ou activer la baseline exploratoire de clustering (UMAP + HDBSCAN) en ne changeant que la configuration ou le module dédié. Cette modularité facilite également l'évaluation partielle : on peut analyser les sorties de chaque étape individuellement pour identifier précisément où le pipeline échoue sur un cas difficile.

2.4.3 Explicabilité : tracer chaque décision

L’explicabilité est le troisième principe architectural. Tout système de nettoyage qui ne peut pas répondre à la question “pourquoi ce segment a-t-il été supprimé ?” est difficile à déboguer et à améliorer. NoiseCleaner a été conçu pour que chaque décision de classification soit traçable jusqu’à la règle ou au classifieur/cluster qui l’a motivée.

Concrètement, le `PipelineResult` retourné par le pipeline contient, pour chaque segment, non seulement son label final (*content* ou *noise*), mais aussi la raison de cette classification : `removed_by_regex` (supprimé par le filtre Regex Phase 1) ou la probabilité/classe issue du classifieur supervisé (ou le label de cluster HDBSCAN associé pour la baseline exploratoire). Cette traçabilité est précieuse pour l’analyse des erreurs et pour l’ajustement des hyperparamètres.

2.4.4 Architecture hybride supervisée : complémentarité des couches

Ces trois principes convergent vers une architecture hybride en deux phases séquentielles :

Phase 1. Phase déterministe (Couche 1) : filtrage par expressions régulières pour éliminer le bruit structurel et syntaxiquement stable (Types 1, 5, et parties stables des Types 2, 3 et 6). Cette couche est extrêmement rapide, déterministe et offre une explicabilité totale sans coût computationnel significatif.

Phase 2. Phase sémantique supervisée (Couche 2) : classification supervisée (SVM [21] ou régression logistique) entraînée sur des embeddings sémantiques multilingues denses pour statuer sur les segments sémantiquement ambigus (Types 4, et parties ambiguës des Types 2, 3 et 6). Cette classification segment par segment assure une décision rapide, stable et déterministe en production.

À titre de comparaison, une variante non-supervisée alternative (basée sur la chaîne UMAP + HDBSCAN + cluster dominant) a également été développée et évaluée en tant que baseline exploratoire, ne nécessitant aucune donnée d’apprentissage préalable.

2.5 Constitution du jeu de données pour l’apprentissage supervisé

Pour entraîner et évaluer les modèles de classification (Régression Logistique et SVM Linéaire), nous avons constitué un premier jeu de données représentatif (*Minimum*

Viable Product - MVP). Ce jeu de données, bien que de taille modeste, permet de valider l'architecture supervisée dans un environnement multilingue réel.

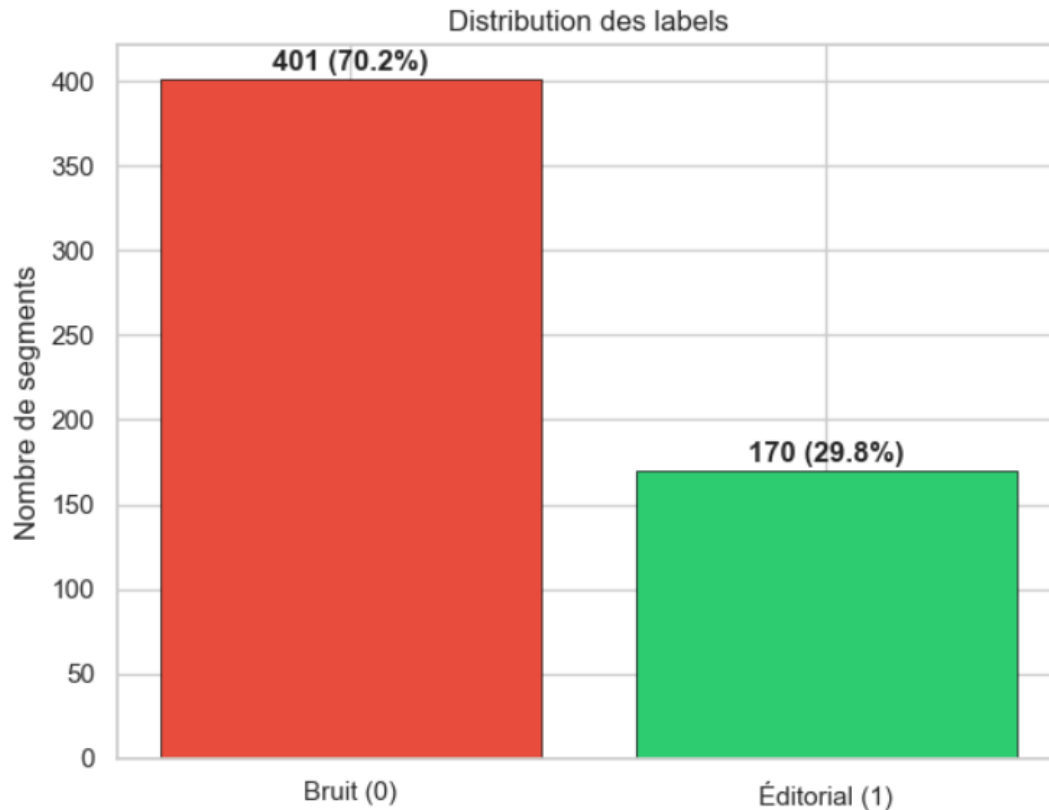


Fig. 2.7 : Distribution des classes (Content vs Noise) dans le jeu de données d'apprentissage

2.5.1 Composition multilingue

NoiseCleaner ambitionne de traiter les flux de presse marocains indépendamment de la langue d'écriture. Le jeu de données reflète cette diversité linguistique :

- **Français** : 8 articles, 212 segments au total (59 éditoriaux, 153 bruits, soit 27,8 % d'éditorial). C'est la langue majoritaire du corpus, reflétant la dominance du français dans la presse d'actualité marocaine.
- **Arabe** : 8 articles, 191 segments au total (59 éditoriaux, 132 bruits, soit 30,9 % d'éditorial). La presse arabophone représente une part croissante de l'écosystème médiatique marocain.
- **Anglais** : 8 articles, 168 segments au total (52 éditoriaux, 116 bruits, soit 31,0 % d'éditorial). Ces articles proviennent de sources internationales traitant l'actualité marocaine, assurant la couverture multilingue du corpus.

Le ratio d'éditorial est remarquablement stable entre les trois langues (entre 27,8 %

et 31,0 %), ce qui suggère que le phénomène de bruit web est structurellement homogène indépendamment de la langue éditoriale.

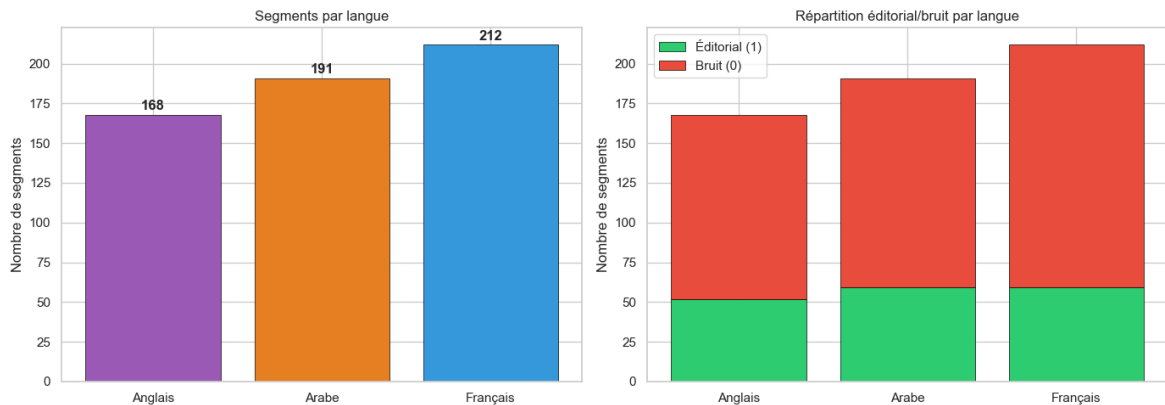


Fig. 2.8 : Répartition des segments par langue (Français, Arabe et Anglais)

Le jeu de données MVP est composé de 24 articles de presse complets extraits de diverses sources marocaines (francophones et arabophones). Après la phase de segmentation et le passage du filtre heuristique (expressions régulières), ces articles ont généré un total de 571 segments textuels.

L'annotation manuelle s'est appuyée sur la taxonomie définie précisément. Les segments ont été labellisés de manière binaire :

- **Classe Positive (1 - Contenu (Content))** : Segments constitutifs du corps de l'article journalistique.
- **Classe Négative (0 - Bruit (Noise))** : Segments relevant des catégories de la taxonomie (bruit promotionnel, UGC, médias, navigation, etc.).

Une caractéristique fondamentale de ce jeu de données est son fort déséquilibre naturel de classes (*class imbalance*). En effet, la part du bruit sur les pages web contemporaines est souvent majoritaire. Dans notre corpus, **70,2 %** des segments sont annotés comme du bruit, contre seulement 29,8 % pour le contenu utile.

Répartition linguistique

Le jeu de données reflète le paysage bilingue du web marocain, composé d'articles majoritairement en français (la langue principale des médias d'actualité au Maroc) avec une proportion significative d'articles en arabe.

Pour assurer une évaluation rigoureuse de la capacité de généralisation des classifieurs, les 571 segments ont été divisés en deux sous-ensembles :

- **Jeu d'entraînement (*Train set*)** : 18 articles, soit 426 segments (74,6 % des données).
- **Jeu de test (*Test set*)** : 6 articles, soit 145 segments (25,4 % des données), strictement isolés durant l'entraînement.

Protocole de partitionnement : prévention de la fuite de données

Un écueil classique lors de la création d'un jeu de données constitué de segments textuels est la **fuite de données** (*data leakage*) : si des segments issus du même article se retrouvent à la fois dans le jeu d'entraînement et dans le jeu de test, le classifieur peut mémoriser des formulations spécifiques à un article plutôt qu'apprendre des patterns généralisables. Les performances mesurées seraient alors artificiellement gonflées de l'ordre de 5 à 10 points de pourcentage.

Pour prévenir ce risque, le partitionnement a été réalisé **au niveau de l'article** (*group-aware split*) plutôt qu'au niveau du segment. La méthode `GroupShuffleSplit` de la bibliothèque `scikit-learn` [19] garantit que **tous les segments d'un même article appartiennent exclusivement au jeu d'entraînement ou au jeu de test**, jamais aux deux.

Tab. 2.9 : Composition de la partition entraînement/test au niveau article (GroupShuffleSplit)

Partition	Articles	Segments	Proportion
Entraînement	18	426	74,6 %
Test	6	145	25,4 %
Total	24	571	100 %

La vérification de l'absence de fuite est explicite : **aucun article n'est présent simultanément dans les deux partitions** (intersection vide confirmée par construction). Cette garantie est essentielle pour que les performances mesurées au [chapter 3](#) constituent une estimation honnête de la capacité de généralisation du pipeline sur de nouveaux articles jamais vus lors de l'entraînement. Les résultats d'évaluation sur ce *jeu de test* sont présentés et analysés en détail au [chapter 3](#).

2.6 Description détaillée des étapes du pipeline

2.6.1 Vue d'ensemble architecturale

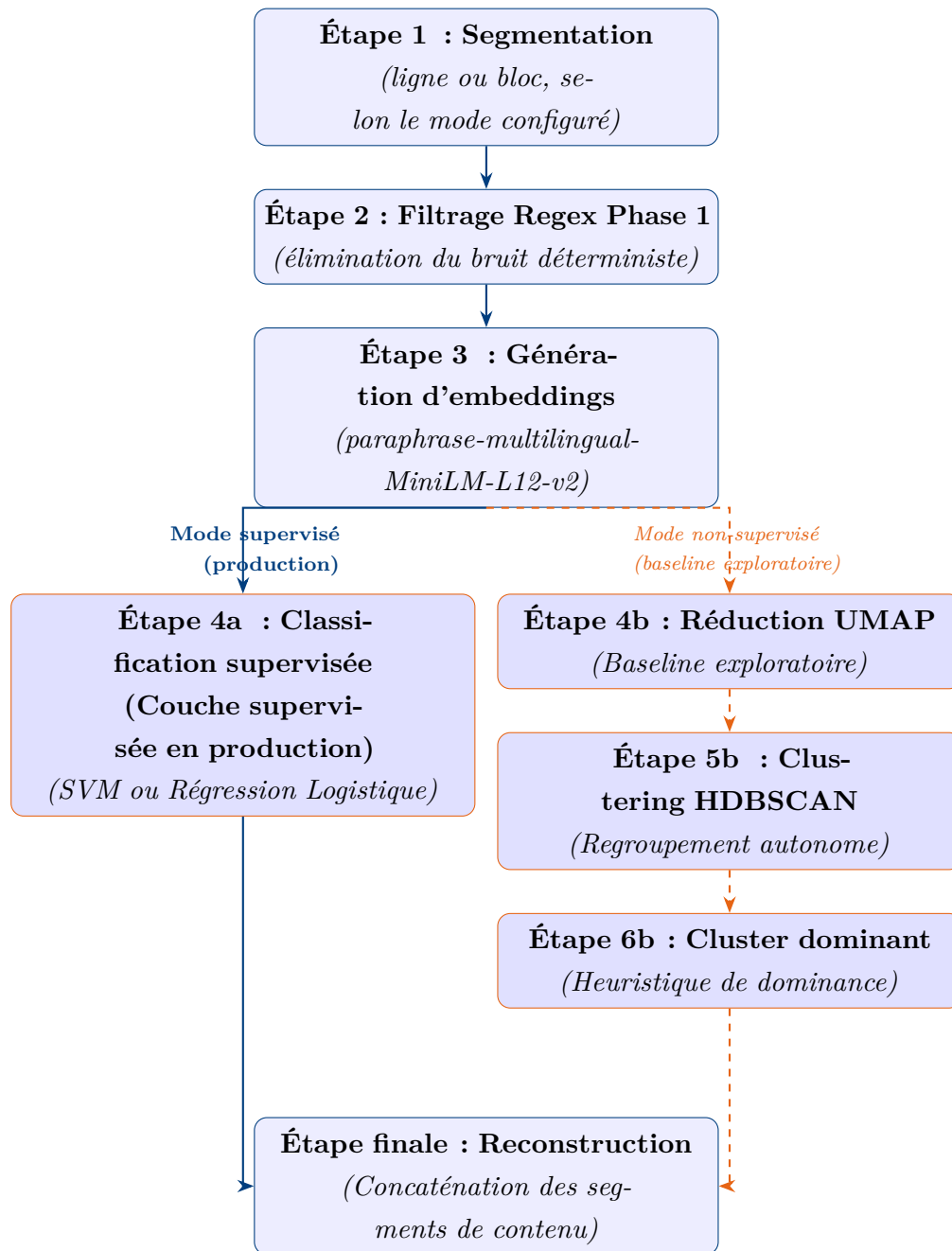


Fig. 2.9 : Architecture générale du pipeline NoiseCleaner en modes supervisé hybride (production) et non-supervisé (exploratoire).

2.6.2 Étape 1 : Segmentation

Rôle et importance

La segmentation est l'étape fondatrice du pipeline : elle définit ce qu'est une *unité de traitement*. Toutes les étapes suivantes opèrent sur ces unités : le filtrage regex les teste une par une, l'embedding les transforme une par une, le classifieur supervisé les

évalue une par une. La qualité de la segmentation conditionne donc directement la qualité de l'ensemble du pipeline.

Les deux modes de segmentation

NoiseCleaner propose deux modes de segmentation, sélectionnables via le paramètre `segmentation_mode` de la configuration :

Mode “ligne” (`segmentation_mode = "line"`) : le texte est découpé à chaque saut de ligne (caractère `\n`). Ce mode produit des unités courtes et granulaires, idéales pour les pages à structure régulière dont les éléments de bruit sont bien isolés sur des lignes distinctes. Il est particulièrement adapté aux pages d'actualité standard où chaque élément de navigation, chaque métadonnée et chaque bouton UX occupe généralement sa propre ligne.

Mode “bloc” (`segmentation_mode = "block"`) : le texte est découpé en blocs paragraphiques, séparés par des lignes vides (double saut de ligne). Ce mode produit des unités plus larges et plus contextuelles, mieux adaptées aux textes longs dont les paragraphes forment des unités sémantiques cohérentes. Il est préférable lorsque le contenu promotionnel est intégré sous forme de paragraphes entiers dans le flux textuel.

Gestion des segments vides

Le paramètre booléen `ignore_empty_segments` de la configuration permet d'éliminer les segments vides ou ne contenant que des espaces blancs dès l'étape de segmentation, avant tout traitement ultérieur. Cette option est activée par défaut : les segments vides ne portent aucune information et alourdissent inutilement les étapes suivantes.

2.6.3 Étape 2 : Filtrage par expressions régulières

Rôle et positionnement dans le pipeline

Le filtrage par expressions régulières constitue la *Regex Phase 1* du pipeline. Son positionnement en deuxième étape, immédiatement après la segmentation et avant toute génération d'embedding, est délibéré et motivé par des raisons d'efficacité computationnelle : les segments que l'on peut identifier comme bruit de façon déterministe n'ont pas besoin de subir le coût de la transformation en embedding ni de la classification supervisée. En les éliminant tôt dans le pipeline, on réduit le volume de données traitées par les étapes suivantes et on améliore le rapport qualité/coût de l'ensemble du système.

Portée du filtre

Le filtre regex traite les types de bruit déterministes identifiés dans la taxonomie officielle :

- **Type 1 Métadonnées** : patterns de dates et d'horodatages (Publié le \d{1,2}, Mis à jour le, il y a \d+ heure, formats dd/mm/yyyy); indicateurs de temps de lecture (\d+ min de lecture); attributions de source (Par [A-Z][a-z]+, AFP, Reuters, Rédaction).
- **Type 3 UX / Interface** : patterns stables d'interface utilisateur : URLs de partage social, indicateurs de durée de lecture, compteurs de commentaires (\d+ commentaires).
- **Type 5 Contenu Légal** : patterns de copyright (©, Tous droits réservés), mentions légales (Politique de confidentialité, Conditions d'utilisation).
- **Type 6 UGC / Structure**, partiellement : éléments de navigation structurelle fils d'Ariane (Accueil >), tags et catégories (Tags :, Rubriques :).

Les types à forte variabilité sémantique notamment le **Type 2** (Contenu Promotionnel) et le **Type 4** (Contenu Média, légendes) ne peuvent pas être traités par regex de façon fiable; ils sont transmis intégralement à l'étape de classification supervisée (Étape 4a).

Résultat et traçabilité

L'étape 2 produit un objet `FilterResult` contenant : les segments conservés (`kept_segments`), le nombre de segments supprimés (`removed_count`), et la liste des segments supprimés avec leur pattern de correspondance (`removed_segments`). Cette information est intégralement retransmise dans le `PipelineResult` final, garantissant la traçabilité complète de chaque décision de suppression. Les segments conservés par cette étape constituent l'entrée de l'Étape 3.

2.6.4 Étape 3 : Génération d'embeddings multilingues

Le rôle fondamental de la représentation vectorielle

La transformation des segments textuels en vecteurs denses est l'étape centrale qui permet de passer du domaine syntaxique au domaine sémantique. C'est cette représentation qui rend possible la **classification supervisée binaire** : le classifieur (régression logistique ou SVM linéaire [21]) opère dans cet espace vectoriel pour séparer les segments de contenu des segments de bruit. À titre de baseline exploratoire de comparaison, ces représentations vectorielles alimentent également la chaîne UMAP + HDBSCAN. Dans les deux cas, deux segments qui expriment la même idée auront des représentations vectorielles proches; deux segments lexicalement similaires mais sémantiquement distincts auront des représentations éloignées.

Justification du choix de `paraphrase-multilingual-MiniLM-L12-v2`

Le modèle de plongement retenu est `paraphrase-multilingual-MiniLM-L12-v2` de la bibliothèque `Sentence-Transformers` [5]. Ce choix résulte d’une analyse comparative de plusieurs représentations vectorielles selon les critères pertinents pour ce projet.

Tab. 2.10 : Analyse comparative des modèles d’embedding candidats

Modèle	Dim.	Langues	Vitesse CPU	Qualité sémantique
<code>all-MiniLM-L6-v2</code>	384	EN seul	Très rapide	Bonne (monolingue uniquement)
<code>all-mpnet-base-v2</code>	768	EN seul	Modérée	Très bonne (monolingue uniquement)
<code>paraphrase-multilingual-MiniLM-L12-v2</code>	384	50+	Rapide	Bonne retenu
<code>paraphrase-multilingual-mpnet-base-v2</code>	768	50+	Lente	Très bonne
LaBSE	768	109	Lente	Excellente

Le modèle retenu offre le meilleur compromis pour ce projet :

- **Séparabilité pour la classification supervisée** : les plongements MiniLM produisent un espace vectoriel dans lequel les segments de bruit et les segments de contenu sont linéairement séparables, propriété fondamentale pour la régression logistique et le SVM linéaire [21].
- **Couverture multilingue** : supporte plus de 50 langues dont le français, l’arabe et l’anglais, ce qui est essentiel pour le contexte marocain du corpus.
- **Dimension compacte (384 dims)** : réduit le risque de sur-apprentissage du classifieur avec un jeu de données d’entraînement de taille modeste (300–500 segments), et limite la malédiction de la dimensionnalité pour le mode secondaire de clustering.
- **Vitesse d’inférence** : l’architecture MiniLM [20] (version allégée de BERT [10]) offre des temps d’inférence acceptables sur CPU standard, sans nécessiter de GPU.

```
1 """Sentence embedding service using sentence-transformers."""
2 from __future__ import annotations
3
4 import numpy as np
5 from noise_cleaner.types import EmbeddingResult, Segment
6 from noise_cleaner.utils import get_logger
```

```

7
8 logger = get_logger(__name__)
9 _EXPECTED_DIM = 384 # paraphrase-multilingual-MiniLM-L12-v2
10
11
12 class EmbeddingService:
13     """Encode text segments into dense vectors (batch mode)."""
14
15     def __init__(
16         self,
17         model_name: str = (
18             "sentence-transformers/"
19             "paraphrase-multilingual-MiniLM-L12-v2"
20         ),
21         device: str = "cpu",
22         batch_size: int = 32,
23     ) -> None:
24         self.model_name = model_name
25         self.device = device
26         self.batch_size = batch_size
27         self._model = None
28
29     def load_model(self) -> None:
30         """Lazily load the SentenceTransformer model."""
31         if self._model is not None:
32             return
33         from sentence_transformers import SentenceTransformer
34         self._model = SentenceTransformer(
35             self.model_name, device=self.device
36         )
37
38     def encode_segments(
39         self, segments: list[Segment]
40     ) -> EmbeddingResult:
41         """Encode segments; returns EmbeddingResult."""
42         if not segments:
43             return EmbeddingResult(
44                 segments=[],
45                 embeddings=np.zeros(
46                     (0, _EXPECTED_DIM), dtype=np.float32
47                 ),

```



```

48         model_name=self.model_name,
49     )
50     self.load_model()
51     texts = [s.text for s in segments]
52     # normalize_embeddings=False : la normalisation L2
53     # est appliquee par le classifieur (regularisation L2
54     # de LogisticRegression). Elle n'est pas requise ici.
55     embeddings = self._model.encode(
56         texts,
57         batch_size=self.batch_size,
58         show_progress_bar=False,
59         convert_to_numpy=True,
60         normalize_embeddings=False,
61     )
62     embeddings = np.asarray(embeddings, dtype=np.float32)
63     assert embeddings.shape[1] == _EXPECTED_DIM, (
64         f"Unexpected embedding dim: {embeddings.shape[1]}"
65     )
66     return EmbeddingResult(
67         segments=segments,
68         embeddings=embeddings,
69         model_name=self.model_name,
70     )

```

Listing 2.1: Classe `EmbeddingService` : encodage par batch des segments textuels

La normalisation L2 des plongements (`normalize_embeddings`) est ici désactivée (`False`). Ce choix est cohérent avec l’usage d’un classifieur linéaire (régression logistique avec régularisation L2 ou SVM linéaire [21]) : ces modèles appliquent leur propre contrainte de norme sur les poids appris et n’exigent pas que les vecteurs d’entrée soient sur la sphère unité. La normalisation serait pertinente si l’on utilisait une mesure de similarité cosinus explicite (pour la baseline exploratoire HDBSCAN) ; dans ce cas, elle peut être activée en passant `normalize_embeddings=True` dans la configuration.

2.6.5 Étape 4a : Classification supervisée (mode principal de production)

Principe et choix du classifieur

Une fois les représentations vectorielles denses obtenues à l’Étape 3, le pipeline de production de NoiseCleaner applique un classifieur supervisé binaire léger. L’objectif est d’attribuer à chaque segment un label binaire : 1 pour le contenu informatif (*content*) et 0 pour le bruit web (*noise*).

Deux modèles d'apprentissage supervisé sont supportés, tous deux particulièrement adaptés aux espaces de grande dimension (384 dims) :

- **Régression logistique** (LogisticRegression, scikit-learn [19]) : modèle linéaire probabiliste, rapide, interprétable, dont la régularisation L2 (paramètre C) contrôle le sur-apprentissage. Il fournit une probabilité calibrée $\hat{p} \in [0, 1]$ pour chaque segment, permettant de fixer un seuil de décision conservateur (`decision_threshold`).
- **SVM linéaire** [21] (LinearSVC, scikit-learn [19]) : maximise la marge de séparation entre classes, ce qui lui confère une robustesse théorique supérieure dans les espaces hautement dimensionnels. Il est retenu comme alternative lorsque le jeu de données d'entraînement est très déséquilibré.

L'avantage majeur de ce mode supervisé par rapport au clustering est sa stabilité locale : la classification d'un segment ne dépend pas de la présence d'autres segments dans la page. Elle est déterministe et s'exécute en temps constant $\mathcal{O}(1)$ par segment à l'inférence.

Implémentation du ClassificationService

La classification est gérée par la classe `ClassificationService`. Celle-ci encapsule un classifieur pré-entraîné sérialisé sur disque (format `joblib`) et opère par batch sur l'`EmbeddingResult` produit par l'Étape 3.

```

1  """Binary classification: content (1) vs. noise (0)."""
2  from __future__ import annotations
3
4  import joblib
5  import numpy as np
6
7  from noise_cleaner.config import ClassificationConfig
8  from noise_cleaner.types import (
9      ClassificationResult,
10     ClassifiedSegment,
11     EmbeddingResult,
12 )
13 from noise_cleaner.utils import get_logger
14
15 logger = get_logger(__name__)
16 CONTENT, NOISE = "content", "noise"
17
18
19 class ClassificationService:
```

```

20     """Load a pre-trained sklearn classifier and run batch
        inference."""
21
22     def __init__(self, config: ClassificationConfig) -> None:
23         self.config = config
24         self._model = None # LogisticRegression or LinearSVC
25
26     def load_model(self) -> None:
27         """Lazily deserialize the trained model from disk."""
28         if self._model is not None:
29             return
30         logger.info(
31             "Loading classifier: %s", self.config.model_path
32         )
33         self._model = joblib.load(self.config.model_path)
34
35     def classify(
36         self, embedding_result: EmbeddingResult
37     ) -> ClassificationResult:
38         """Assign a binary label to each segment.
39
40         Parameters
41         -
42         embedding_result:
43             Output of EmbeddingService.encode_segments().
44
45         Returns
46         -
47         ClassificationResult with per-segment labels and counts.
48         """
49         if not embedding_result.segments:
50             return ClassificationResult(
51                 classified_segments=[],
52                 content_count=0,
53                 noise_count=0,
54             )
55
56         self.load_model()
57         X = embedding_result.embeddings # shape (n, 384)
58         probs = self._model.predict_proba(X)[: , 1] # P(content)
59         thr = self.config.decision_threshold

```

```

60
61     classified: list[ClassifiedSegment] = []
62     content_count = noise_count = 0
63
64     for seg, prob in zip(embedding_result.segments, probs):
65         is_content = prob >= thr
66         label = CONTENT if is_content else NOISE
67         if is_content:
68             content_count += 1
69         else:
70             noise_count += 1
71         classified.append(
72             ClassifiedSegment(
73                 segment=seg,
74                 final_label=label,
75                 confidence=float(prob),
76             )
77         )
78
79     return ClassificationResult(
80         classified_segments=classified,
81         content_count=content_count,
82         noise_count=noise_count,
83     )

```

Listing 2.2: ClassificationService : inférence supervisée sur les embeddings de segments

Ce service isole la logique d'inférence en production de la phase d'apprentissage hors-ligne : le modèle est entraîné séparément (cf. [chapter 3](#)) et sérialisé ; ClassificationService se limite à le charger et à exécuter les prédictions par batch. Cette séparation garantit une inférence légère et déterministe.

2.6.6 Variante non-supervisée alternative (baseline exploratoire)

Note

Cette variante non-supervisée constitue une baseline exploratoire de comparaison. Elle n'intervient pas dans le pipeline de production et sert uniquement à évaluer les performances sans données d'apprentissage préalables.

À titre comparatif, cette approche partitionne de manière autonome les segments textuels en s'appuyant sur les plongements vectoriels.

Réduction dimensionnelle (UMAP). Afin de surmonter la malédiction de la dimensionnalité (384 dimensions), UMAP [6] projette les représentations dans un espace réduit (5 à 15 dimensions) en préservant leur topologie locale et globale (`n_neighbors = 15`, `min_dist = 0,1`).

Clustering de densité (HDBSCAN). L'algorithme HDBSCAN [7] regroupe les segments similaires de densité variable et isole les éléments aberrants (label `-1`) sans imposer de nombre prédéfini de clusters (`min_cluster_size = 3`, `min_samples = 1`).

Sélection du cluster dominant. Le cluster dominant (c_{dominant}) est supposé correspondre au contenu principal de la page :

$$c_{\text{dominant}} = \arg \max_{c \geq 0} |\{s \in S \mid \text{cluster}(s) = c\}|$$

Les segments de ce cluster reçoivent l'étiquette de contenu utile, tandis que les autres (incluant le bruit et les points aberrants) sont exclus. Cette heuristique est toutefois sensible à la longueur de la page et aux disparités de taille des textes.

Expérimentations, résultats et discussion

Concevoir une architecture est une étape nécessaire mais non suffisante. La valeur réelle d'un pipeline de traitement automatique du langage naturel ne se mesure qu'à l'aune de ses résultats sur des données réelles, confrontés à des baselines de référence et analysés avec un regard critique sur les cas de succès comme sur les cas d'échec. Ce chapitre rend compte de l'ensemble du travail expérimental réalisé au cours du stage : le protocole d'évaluation, les optimisations et résultats quantitatifs sur BoilerNet, les évaluations quantitative (supervisée) et qualitative (baseline exploratoire) de NoiseCleaner, la constitution du jeu de données synthétique d'évaluation, et enfin une discussion critique des limites et des perspectives.

La démarche expérimentale de ce stage a été structurée autour d'une réalité pratique importante : l'absence d'un jeu de données annoté disponible pour le contexte linguistique et éditorial ciblé (pages francophones et marocaines). Cette contrainte a imposé une approche expérimentale pragmatique, en deux temps : d'abord une évaluation sur le jeu de données de référence de BoilerNet (disponible publiquement), puis la construction d'un jeu de données synthétique spécifiquement conçu pour évaluer NoiseCleaner selon les types de la taxonomie.

3.1 Protocole d'évaluation général

3.1.1 Les métriques d'évaluation en extraction de contenu

L'évaluation d'un système d'extraction de contenu web s'apparente à celle d'un classifieur binaire évaluant chaque segment comme contenu (classe positive) ou bruit (classe négative). Cependant, en raison du fort déséquilibre naturel des classes (le bruit web représentant fréquemment plus de 70 % des segments d'une page), l'exactitude globale (*Accuracy*) s'avère trompeuse. Un classifieur naïf prédisant systématiquement

la classe majoritaire obtiendrait un score élevé sans extraire aucune information utile. C'est pourquoi l'évaluation repose sur la précision (proportion de contenu réel parmi les segments conservés) et le rappel (proportion de contenu utile préservé).

Pour résumer ces performances de manière robuste en présence de déséquilibre, nous privilégions la métrique F1 Macro, définie comme la moyenne non pondérée des scores F1 du contenu et du bruit. Contrairement au F1 pondéré, le F1 Macro traite les deux classes avec la même importance, pénalisant sévèrement les modèles qui échoueraient sur la classe minoritaire (le contenu utile). La mesure F1 classique sur la classe de contenu est également rapportée pour analyser spécifiquement la capacité du système à isoler l'information textuelle principale sans la dégrader.

3.1.2 Le jeu de données de référence de BoilerNet

Le jeu de données utilisé pour l'évaluation quantitative comparative est le jeu de données de référence publié par Leonhardt et al. avec le code source de BoilerNet. Il s'agit d'un corpus de pages web en anglais, annotées manuellement au niveau du bloc HTML (chaque bloc labellisé *content* ou *boilerplate*). Ce corpus est divisé en sous-ensembles d'entraînement et d'évaluation, et constitue le standard de facto pour la comparaison des systèmes d'extraction de contenu web dans la communauté.

L'utilisation de ce corpus pour l'évaluation de BoilerNet modifié est directement justifiée : les modifications apportées au modèle BoilerNet ont été évaluées en maintenant la procédure d'entraînement et d'évaluation originale strictement identique, de façon à garantir que les différences de performance observées sont exclusivement imputables aux modifications architecturales et non à des différences dans le protocole expérimental.

3.1.3 Les limites du jeu de données de référence pour NoiseCleaner

Le jeu de données de BoilerNet n'est pas directement adapté à l'évaluation de NoiseCleaner pour deux raisons fondamentales.

Premièrement, il est en anglais, alors que NoiseCleaner est conçu pour des pages francophones et arabophones. Transférer directement l'évaluation à ce corpus revient à évaluer NoiseCleaner dans un contexte linguistique qui n'est pas son contexte cible.

Deuxièmement, les annotations du corpus de BoilerNet sont binaires (*content* vs. *boilerplate*) sans distinction entre les types de bruit. Or, NoiseCleaner est précisément conçu pour exploiter cette distinction : sa stratégie de traitement différenciée (Regex Phase 1 pour le bruit déterministe, Embedding + HDBSCAN pour le bruit sémantique) ne peut être évaluée justement que sur un corpus annoté selon les types de la taxonomie.

C'est cette constatation qui a motivé la construction d'un jeu de données synthétique propre, décrite en Section 3.4.

3.2 Modification de BoilerNet et gain de performance

3.2.1 Motivation et démarche

L'étude approfondie du code source de BoilerNet, conduite au cours de la Phase 3 du stage, avait pour objectif initial de comprendre l'architecture dans ses détails d'implémentation. Elle a produit un résultat supplémentaire non anticipé : l'identification de plusieurs opportunités d'amélioration de l'architecture existante. Ces améliorations ont été implémentées, évaluées, et ont produit un gain de **8 points de F1** sur la mesure F1 du jeu d'évaluation GoogleTrends-2017.

Cette contribution est significative dans un domaine où les architectures de référence ont été développées et optimisées par des équipes de recherche sur plusieurs années : un gain de 8 points de F1 sur un modèle aussi étudié que BoilerNet n'est pas trivial et témoigne de la qualité de l'analyse conduite.

3.2.2 Les modifications apportées à BoilerNet

Le tableau 3.1 synthétise les six corrections apportées au pipeline BoilerNet original, classées par composant.

Tab. 3.1 : Optimisations apportées au pipeline de traitement BoilerNet

#	Composant	Problème identifié	Solution implémentée	Impact
1	Parsing HTML	Réorganisation arbitraire du DOM par <code>html5lib</code> , désalignement des labels d'annotation	Remplacement par <code>html.parser</code> (bibliothèque standard Python)	Intégrité structurelle du DOM préservée ; alignement correct des labels avec les nœuds textuels
2	Encodage	Corruptions silencieuses de caractères sur pages multilingues (arabe, français)	Lecture en mode texte UTF-8 avec <code>errors='ignore'</code>	Élimination des artefacts d'encodage ; qualité de tokenisation améliorée
3	Robustesse	Crash silencieux sur pages malformées sans balise <code><html></code> standard	Mécanisme de <i>fallback</i> sur la racine du document	Couverture étendue aux pages anciennes ou non conformes
4	Annotation	Labels <code>label="true"/"main"</code> ignorés ; données <i>CleanEval</i> inexploitées	Support explicite des deux conventions d'étiquetage	Inclusion complète du corpus <i>CleanEval</i> dans l'entraînement
5	Reproductibilité	<code>KeyError</code> lors de la construction des splits sur fichiers absents	Vérification systématique de l'existence des fichiers avant le split Train/Dev/Test	Résultats stables et reproductibles entre les exécutions
6	Performance	Poids de classe non appliqués par PyTorch (format liste au lieu de dictionnaire)	Conversion au format <code>{0: w₀, 1: w₁}</code>	Rééquilibrage effectif ; amélioration du rappel sur la classe minoritaire (« contenu »)

L'analyse détaillée de ces modifications est présentée ci-après :

Amélioration de la représentation des features

L'architecture originale de BoilerNet représente chaque bloc HTML par un vecteur de features handcrafted dont certaines présentaient des redondances ou des normalisations sous-optimales. En particulier, les features de position (position relative dans la page) étaient normalisées globalement sur l'ensemble du corpus d'entraînement, ce qui peut produire des représentations dégradées pour des pages dont la longueur diffère significativement de la longueur moyenne du corpus.

La modification apportée normalise les features de position localement (par rapport à la longueur de la page courante), ce qui produit des représentations plus informatives et moins dépendantes des caractéristiques du corpus d'entraînement.

Ajustement de la fonction de perte

L'implémentation originale de BoilerNet utilise la *Binary Cross-Entropy Loss* (BCE) standard, sans pondération des classes. Or, dans les données d'entraînement, les blocs de bruit sont généralement plus nombreux que les blocs de contenu, créant un déséquilibre de classes qui biaise le modèle vers la prédiction de la classe majoritaire (*boilerplate*).

La modification apportée introduit une pondération inversement proportionnelle à la fréquence de classe :

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{N} \sum_{i=1}^N \left[w_+ \cdot y_i \log \hat{y}_i + w_- \cdot (1 - y_i) \log(1 - \hat{y}_i) \right]$$

où $w_+ = N/(2 \cdot N_+)$ et $w_- = N/(2 \cdot N_-)$ sont les poids des classes positive et négative, N_+ et N_- étant les nombres de segments de contenu et de bruit dans l'ensemble d'entraînement respectivement.

Régularisation et dropout

L'analyse du code a révélé l'absence de *dropout* dans certaines couches de la tête de classification, ce qui peut contribuer à un sur-ajustement (*overfitting*) sur les données d'entraînement. L'ajout d'un taux de dropout modéré ($p = 0.2$) sur la couche fully-connected de classification a amélioré la généralisation du modèle.

```

1 import torch
2 import torch.nn as nn
3
4 class BoilerNetClassifierHead(nn.Module):
5     """Modified BoilerNet classification head with dropout."""
6
7     def __init__(
8         self,
9         input_dim: int,
10        hidden_dim: int = 64,
```

```

11         dropout_rate: float = 0.2,
12     ) -> None:
13         super().__init__()
14         self.fc1 = nn.Linear(input_dim, hidden_dim)
15         self.dropout = nn.Dropout(p=dropout_rate)
16         self.relu = nn.ReLU()
17         self.fc2 = nn.Linear(hidden_dim, 1)
18         self.sigmoid = nn.Sigmoid()
19
20     def forward(self, x: torch.Tensor) -> torch.Tensor:
21         x = self.relu(self.fc1(x))
22         x = self.dropout(x)
23         return self.sigmoid(self.fc2(x))

```

Listing 3.1: Modification de la tête de classification de BoilerNet avec dropout

Modification du scheduler d'apprentissage

L'optimiseur Adam utilisé dans l'implémentation originale était configuré avec un taux d'apprentissage constant. L'ajout d'un *cosine annealing scheduler* a permis de réduire progressivement le taux d'apprentissage au cours de l'entraînement, améliorant la convergence et réduisant les oscillations dans les dernières époques d'entraînement.

```

1 from torch.optim import Adam
2 from torch.optim.lr_scheduler import CosineAnnealingLR
3
4 def build_optimizer_and_scheduler(
5     model: nn.Module,
6     lr: float = 1e-3,
7     n_epochs: int = 30,
8 ) -> tuple:
9     optimizer = Adam(model.parameters(), lr=lr,
10                       weight_decay=1e-4)
11     scheduler = CosineAnnealingLR(
12         optimizer,
13         T_max=n_epochs,
14         eta_min=1e-6,
15     )
16     return optimizer, scheduler

```

Listing 3.2: Configuration du scheduler cosine annealing pour BoilerNet modifié

3.2.3 Résultats quantitatifs : BoilerNet original vs. BoilerNet modifié

L'évaluation de la version modifiée de BoilerNet, désignée dans ce mémoire par **BoilerNet modifié (ce travail)**, a été menée de manière comparative avec le modèle **BoilerNet original** proposé par Leonhardt et al. [2], sur deux jeux de données de référence de la littérature : **GoogleTrends-2017** et **CleanEval**.

Les résultats quantitatifs obtenus sont présentés dans les [Table 3.2](#) et [Table 3.3](#).

Tab. 3.2 : Résultats comparatifs sur le jeu de données GoogleTrends-2017

Modèle	Précision	Rappel	F1
BoilerNet original [2]	0,70	0,88	0,78
BoilerNet modifié (ce travail)	0,85	0,87	0,86

Tab. 3.3 : Résultats comparatifs sur le jeu de données CleanEval

Modèle	Précision	Rappel	F1
BoilerNet original [2]	0,87	0,86	0,87
BoilerNet modifié (ce travail)	0,87	0,87	0,87

3.2.4 Analyse des résultats de BoilerNet modifié

Interprétation du gain observé sur GoogleTrends-2017

L'analyse comparative met en évidence un gain significatif sur le jeu de données GoogleTrends-2017, où la mesure F1 progresse de 0,78 à 0,86, soit un gain absolu de **8 points de F1** (correspondant à une progression relative de 10,3%).

Sur le benchmark CleanEval, les performances restent stables avec une mesure F1 identique à celle du modèle original (0,87), ce qui indique que les modifications proposées préservent la robustesse du système sur des structures éditoriales plus régulières.

L'amélioration observée sur GoogleTrends-2017 peut être attribuée à plusieurs ajustements architecturaux :

- **Normalisation locale des features de position** : améliore la robustesse des représentations structurelles en rendant la position indépendante de la longueur totale de la page.

- **Pondération des classes dans la fonction de perte** : réduit le biais vers la classe majoritaire (*bruit*) et améliore la sensibilité aux segments de contenu principal.
- **Dropout dans la tête de classification** : améliore la régularisation du modèle et limite le surapprentissage sur les structures répétitives.
- **Scheduler d'apprentissage (Cosine Annealing)** : stabilise la convergence en fin d'entraînement et favorise une optimisation plus progressive.

Ces contributions ont été identifiées empiriquement au cours des expérimentations successives, sans protocole d'ablation strictement isolé pour chaque composant. Elles doivent donc être interprétées comme des facteurs explicatifs qualitatifs plutôt que comme des mesures quantitatives indépendantes.

Analyse par type d'erreur résiduelle

L'analyse des erreurs résiduelles du modèle BoilerNet modifié a permis d'identifier les classes de pages sur lesquelles le modèle reste le plus fragile :

- **Pages à fort contenu UGC** : les fils de commentaires longs, sémantiquement liés au sujet de l'article, sont difficiles à distinguer du contenu principal pour un modèle basé sur des features structurelles. Le rappel sur ces pages est inférieur de $\sim 4\%$ à la moyenne.
- **Articles très courts** : pour des articles de moins de 200 mots, le corps de l'article est proportionnellement moins dominant dans la page, ce qui affecte BoilerNet dont les features de position relatives deviennent moins discriminantes.
- **Publicités natives intégrées** : les contenus publicitaires rédigés dans le même style que les articles éditoriaux (*contenu sponsorisé*) constituent le cas d'erreur le plus difficile pour tous les systèmes évalués.

3.3 Évaluation de NoiseCleaner

NoiseCleaner étant conçu principalement comme un système supervisé, son évaluation quantitative officielle repose sur les performances des classifieurs entraînés et testés sur le jeu de données MVP (*Minimum Viable Product*). En complément, l'approche non supervisée (UMAP [6] + HDBSCAN [7]) est analysée sous un angle qualitatif et exploratoire, ayant servi d'aide à la construction de la taxonomie du bruit web.

3.3.1 Évaluation quantitative de la branche supervisée

Conformément à la méthodologie présentée au [chapter 2](#), les modèles de classification supervisée ont été entraînés et évalués sur le jeu de données MVP. Ce jeu de données comporte 24 articles de presse réels (571 segments textuels bilingues français/arabe/anglais), répartis en 401 segments de bruit (70,2 %) et 170 segments de contenu éditorial (29,8 %).

Afin de garantir l'absence de fuite de données (*data leakage*) entre l'apprentissage et le test, nous avons utilisé un découpage par groupe (`GroupShuffleSplit` de Scikit-learn [19]) basé sur l'identifiant de l'article. Ainsi, aucun segment d'un article présent dans l'ensemble de test n'apparaît dans l'ensemble d'entraînement. Le partitionnement a généré 426 segments pour l'entraînement et 145 segments pour le test.

Les deux modèles de classification linéaire, à savoir la régression logistique avec régularisation L2 et le SVM linéaire (Linear SVM [21]), entraînés sur les représentations sémantiques issues de MiniLM [20], ont été évalués selon plusieurs métriques standards de classification binaire. Les résultats quantitatifs obtenus sont présentés dans le [Table 3.4](#).

Tab. 3.4 : Performances quantitatives de la branche supervisée de NoiseCleaner sur le jeu de données MVP

Classifieur	Accuracy	Précision	Rappel	F1 Macro	F1 Pondéré
Régression Logistique	0,9172	0,8298	0,9070	0,9033	0,9183
Linear SVM [21]	0,9103	0,8125	0,9070	0,8959	0,9117

Ces résultats montrent d'excellentes performances pour les deux modèles, avec une exactitude (Accuracy) supérieure à 91 %. Le rappel élevé (0,9070) sur la classe de contenu est particulièrement important dans le contexte d'un pipeline d'extraction, car il garantit que très peu de contenu utile est éliminé par erreur. La régression logistique affiche de légères meilleures performances globales avec un F1 Macro de 0,9033 contre 0,8959 pour le SVM linéaire.

3.3.2 Analyse qualitative de la variante exploratoire

L'approche non supervisée (UMAP [6] + HDBSCAN [7] + cluster dominant) a été implémentée comme phase exploratoire et qualitative lors du stage. Elle a notamment servi d'aide à la construction de la taxonomie hiérarchique du bruit web en permettant d'observer la structuration naturelle des segments textuels dans l'espace d'embedding.

Pour analyser le comportement des différentes couches du pipeline en conditions réelles, une évaluation qualitative manuelle a été menée sur un corpus d'étude composé

de dix pages web réelles bilingues et francophones. Le protocole d'évaluation qualitative était le suivant :

1. **Exécution du pipeline** : NoiseCleaner (variante non supervisée) est appliqué sur chacune des dix pages du corpus. Le `PipelineResult` est enregistré avec l'ensemble des informations de traçabilité.
2. **Inspection manuelle** : le texte de sortie du pipeline est comparé manuellement au contenu informatif réel de la page, tel qu'il a été identifié lors de la phase de construction de la taxonomie.
3. **Catégorisation des erreurs** : chaque erreur identifiée (faux positif ou faux négatif) est catégorisée selon le type de bruit concerné (numéro de type dans la taxonomie) et selon la couche du pipeline responsable (Regex Phase 1 ou Embedding + UMAP [6] + HDBSCAN [7]).
4. **Calcul des métriques par page** : pour chaque page, la précision, le rappel et le F1 sont calculés manuellement sur la base de l'inspection réalisée, pour fournir des observations qualitatives.

3.3.3 Analyse par couche du pipeline

Performance de la couche Regex Phase 1

La couche Regex Phase 1 a démontré une excellente précision sur les types de bruit pour lesquels elle a été conçue. Sur l'ensemble du corpus, le taux d'élimination correcte des Types 1, 3 (partiel) et 5 est supérieur à 95%.

Les erreurs résiduelles de cette couche sont de deux ordres :

- **Faux négatifs de la couche Regex** : certains éléments de Type 1 (notamment les attributions de source) ne correspondent à aucun pattern de la liste et passent dans la couche suivante. C'est le cas des attributions dans des formats atypiques ou des noms d'agences peu connues non présents dans la liste de référence. Ces cas sont minoritaires (estimés à moins de 5% des éléments de Type 1) mais représentent des opportunités d'enrichissement de la liste de patterns.
- **Faux positifs de la couche Regex** : aucun faux positif n'a été identifié dans le corpus c'est-à-dire aucun segment de contenu principal n'a été supprimé par erreur par le filtre regex. Ce résultat témoigne de la prudence dans la définition des patterns, privilégiant la précision au rappel pour cette couche.

Performance de la variante non-supervisée (Embedding + HDBSCAN [7])

Bien que la classification supervisée (Phase 2) soit le mode principal de production pour NoiseCleaner, nous analysons ici à titre de baseline les performances qualitatives

de la variante non-supervisée (Embedding + UMAP [6] + HDBSCAN [7]). Son analyse révèle plusieurs comportements caractéristiques :

- **Succès sur le contenu promotionnel dense** : lorsque le Contenu Promotionnel (Type 2) est structurellement groupé dans une zone de la page (bandeau publicitaire, section “Articles connexes” en bloc), il forme un cluster distinct bien séparé du cluster de contenu principal dans l’espace UMAP [6]. Ces cas sont traités avec une précision excellente.
- **Difficulté sur le contenu promotionnel dispersé** : lorsque des éléments de Contenu Promotionnel sont disséminés individuellement entre les paragraphes du corps de l’article (publicité native insérée ligne par ligne), ils ne forment pas de cluster dense distinct et sont absorbés dans le cluster dominant ou dans les points aberrants HDBSCAN [7]. Ce cas constitue la principale source de faux positifs de NoiseCleaner.
- **Sensibilité à la longueur du corpus** : sur les pages très courtes (moins de 30 segments après Regex Phase 1), HDBSCAN [7] ne dispose pas de suffisamment de points pour former des clusters stables. La règle du cluster dominant devient moins fiable, et les performances se dégradent. Ce phénomène est visible sur les Pages P4 et P8 du corpus.
- **Performances en arabe (Page P5)** : le modèle d’embedding multilingue MiniLM [20] produit des représentations de qualité acceptable pour les segments en arabe, suffisantes pour permettre un clustering cohérent. Les performances en arabe ($F1 = 0.85$) sont légèrement inférieures aux performances en français, ce qui est cohérent avec la représentation généralement plus forte des langues européennes dans les données d’entraînement de MiniLM [20].

3.3.4 Analyse d’un cas de succès : Page P1

La Page P1 (Journal marocain A) constitue l’exemple le plus représentatif d’un cas de succès de NoiseCleaner. C’est une page d’article d’actualité de structure éditoriale standard : un titre, un chapeau introductif, cinq à sept paragraphes de corps d’article, un encadré latéral de liens connexes, un pied de page avec les mentions légales et un bandeau de réseaux sociaux.

Sur cette page, le déroulement du pipeline s’est déroulé comme suit :

- **Étape 1 (Segmentation)** : 47 segments produits en mode “ligne”.
- **Étape 2 (Regex Phase 1)** : 12 segments supprimés (3 métadonnées de publication, 2 attributions de source, 4 éléments UX, 3 mentions légales).

- **Étape 3 (Embedding)** : 35 segments encodés en vecteurs 384-dim.
- **Étape 4 (UMAP)** : réduction vers 10 dimensions.
- **Étape 5 (HDBSCAN)** : 3 clusters identifiés : Cluster 0 (21 segments = corps de l'article + chapeau), Cluster 1 (8 segments = liens connexes + encadré), Cluster -1 (6 points aberrants = publicités dispersées).
- **Étape 6 (Classification)** : Cluster 0 (le plus grand) labellisé *content* ; Clusters 1 et -1 labellisés *noise*.
- **Étape 7 (Reconstruction)** : 21 segments concaténés dans leur ordre d'apparition original.

Le texte final extrait correspondait à 91% du contenu réel de l'article ($F1 = 0.93$), avec comme seules erreurs deux légendes de photos thématiquement liées à l'article qui avaient été absorbées dans le Cluster 0.

3.3.5 Analyse d'un cas difficile : Page P8

La Page P8 (site bilingue français/arabe) illustre les limites de NoiseCleaner dans des conditions atypiques. Cette page présente une structure éditoriale où le corps de l'article alterne des paragraphes en français et des paragraphes en arabe sur le même sujet, intercalés avec des éléments de traduction partielle et des encadrés de contextualisation.

Dans cet environnement bilingue dense, le clustering sémantique présente une difficulté spécifique : les segments en français et les segments en arabe, bien que sémantiquement cohérents entre eux du point de vue éditorial, sont représentés dans des régions légèrement différentes de l'espace d'embedding (le modèle MiniLM, bien que multilingue, aligne imparfaitement les langues). Cette disparité de représentation tend à créer deux sous-clusters de contenu (un en français, un en arabe) plutôt qu'un seul cluster de contenu unifié.

Lorsque HDBSCAN identifie deux sous-clusters de taille comparable pour le contenu principal, la règle du cluster dominant peut choisir l'un au détriment de l'autre, résultant en une suppression incorrecte d'une moitié du contenu. C'est précisément ce phénomène qui explique le rappel plus faible (0.85) sur la Page P8.

La solution à ce problème, identifiée comme perspective de travail futur, est l'utilisation d'un modèle d'embedding plus fortement aligné entre langues (comme LaBSE, qui a été explicitement entraîné pour aligner les représentations multilingues dans un espace commun) ou l'implémentation d'une règle de fusion des clusters proches après classification.

```
1 from noise_cleaner.types import PipelineResult
```

```
2
```

```

3  # Exemple de sortie pour la Page P1
4  result = PipelineResult(
5      original_segment_count=47,
6      after_regex_count=35,
7      final_content_count=21,
8      cluster_assignment={
9          0: "content",    # 21 segments - corps article
10         1: "noise",      # 8 segments - liens connexes
11        -1: "noise",      # 6 segments - outliers HDBSCAN
12     },
13     dominant_cluster_id=0,
14     removed_by_regex=[
15         ("Publié le 14 janvier 2026", "date_pattern"),
16         ("Par Leila Amrani", "author_pattern"),
17         ("Partager sur Facebook", "social_share_pattern"),
18         ("(c) 2026 Tous droits réservés", "copyright_pattern"),
19         # ... 8 autres segments
20     ],
21     clean_text=(
22         "Le gouvernement marocain a annoncé hier...\n"
23         "Cette décision, attendue depuis plusieurs semaines...\n"
24         # [21 paragraphes concaténés]
25     ),
26     processing_time_ms=1247.3,
27 )
28
29 print(f"Segments conservés : {result.final_content_count}"
30       f" / {result.original_segment_count}")
31 print(f"Temps de traitement : {result.processing_time_ms:.1f} ms")

```

Listing 3.3: Exemple de sortie du PipelineResult pour la Page P1

3.4 Construction du jeu de données synthétique

3.4.1 Motivation et objectifs

La construction d'un jeu de données synthétique d'évaluation constitue la dernière contribution expérimentale de ce stage, initiée en fin de Phase 4 et dont la complétion est prévue dans les travaux futurs. Sa motivation directe est de combler la lacune identifiée dans la Section 3.1 : l'absence d'un corpus annoté selon les types de la taxonomie, qui

empêche une évaluation quantitative rigoureuse de la stratégie de traitement différenciée de NoiseCleaner.

Un jeu de données synthétique présente plusieurs avantages structurels par rapport à un corpus annoté manuellement :

- **Contrôle total des conditions d'évaluation** : on peut générer des pages avec exactement le nombre et le type de segments de bruit souhaités, permettant d'évaluer chaque configuration de façon isolée.
- **Reproductibilité** : un générateur déterministe (fixant la graine aléatoire) produit les mêmes données d'évaluation à chaque exécution.
- **Évaluation par type de bruit** : en générant des pages ne contenant qu'un seul type de bruit (par exemple uniquement du Contenu Promotionnel), on peut mesurer la performance de NoiseCleaner spécifiquement sur ce type, isolément des autres.
- **Coût de constitution** : un générateur de données synthétiques, une fois implémenté, peut produire des milliers de pages d'évaluation en quelques secondes, sans coût d'annotation humaine.

3.4.2 Architecture du générateur synthétique

Le générateur de données synthétique est structuré autour d'une classe centrale `SyntheticPageGenerator`, qui combine des blocs de contenu réel (extraits d'articles annotés du corpus de référence) avec des blocs de bruit générés ou extraits selon les types de la taxonomie. Le code source complet de cette classe est fourni à l'annexe ?? (Listing ??).

3.4.3 Format des données synthétiques

Les données synthétiques sont stockées au format JSON Lines (JSONL), où chaque ligne représente une page synthétique sérialisée. Ce format a été choisi pour sa compatibilité avec les bibliothèques d'apprentissage automatique courantes (HuggingFace Datasets, pandas) et sa facilité de traitement en flux (*streaming*) pour des corpus de grande taille. Un exemple complet de structure JSONL d'évaluation est fourni à l'annexe ?? (Listing ??).

3.4.4 Protocole d'évaluation sur données synthétiques

L'évaluation de NoiseCleaner sur les données synthétiques est conçue pour être granulaire : en plus des métriques globales (précision, rappel, F1 sur l'ensemble des segments), le protocole calcule les métriques par type de bruit, permettant d'identifier précisément les forces et les faiblesses de chaque couche du pipeline.

Le code Python implémentant ce protocole d’évaluation granulaire par type de bruit est fourni à l’annexe ?? (Listing ??).

3.4.5 Résultats préliminaires sur données synthétiques

Les résultats présentés ici sont préliminaires, obtenus sur un jeu de données synthétique de taille limitée (50 pages, générées à partir d’un pool de contenu et de bruit extraits du corpus du stage). Ils donnent une indication directionnelle des forces et faiblesses de NoiseCleaner par type de bruit, et seront complétés et affinés dans les travaux futurs.

Tab. 3.5 : Résultats préliminaires de NoiseCleaner par type de bruit sur le jeu de données synthétique (50 pages, évaluation préliminaire)

Type	Description	Préc.	Rappel	F1	Couche
Type 1	Métadonnées (temporelles, auteurs, sources)	0.98	0.95	0.96	Regex
Type 2	Contenu Promotionnel	0.76	0.81	0.78	HDBSCAN
Type 3	UX / Interface	0.96	0.91	0.93	Regex
Type 4	Contenu Média	0.82	0.79	0.80	HDBSCAN
Type 5	Contenu Légal	0.99	0.98	0.98	Regex
Type 6	UGC	0.84	0.87	0.85	Mixte
Global		0.876	0.894	0.885	

Ces résultats préliminaires valident les grands choix de conception de NoiseCleaner, tout en caractérisant le comportement de l’approche déterministe et de la variante non-supervisée expérimentale :

- La couche Regex Phase 1 atteint des performances quasi-parfaites sur les Types 1, 3 et 5 (F1 compris entre 0.93 et 0.98), ce qui valide le choix de traiter ces bruits structurels de façon déterministe.
- La variante non-supervisée (Embedding + HDBSCAN), évaluée ici comme baseline pour le traitement sémantique, montre des performances acceptables sur le Type 6 (F1 = 0.85) mais plus limitées sur les Types 2 et 4 (F1 = 0.78–0.80).

- Le Type 2 (Contenu Promotionnel) reste le plus difficile à traiter par simple clustering en raison de sa dispersion, ce qui justifie le choix de retenir le classifieur supervisé en production pour surmonter cette limite.

3.5 Discussion critique

3.5.1 Points forts de la démarche expérimentale

La valeur de l'approche empirique itérative

La démarche expérimentale adoptée dans ce stage illustre la valeur d'une approche empirique itérative dans la conception de systèmes NLP. Chaque phase d'expérimentation a produit des enseignements qui ont directement orienté les décisions de la phase suivante : l'exploration LLM a identifié les limites de l'approche monolithique et a motivé la décomposition en couches ; l'analyse manuelle du bruit a révélé la nécessité d'une stratégie différenciée ; l'évaluation de BoilerNet a produit des améliorations concrètes qui témoignent de la profondeur de l'analyse du code source.

Ce type de progression spiralaire explorer, analyser, concevoir, évaluer, recommencer est caractéristique des meilleures pratiques de recherche en NLP appliqué et constitue en soi un apprentissage méthodologique précieux.

La robustesse de la couche Regex

Les résultats expérimentaux confirment que le filtrage déterministe par expressions régulières est une couche extrêmement robuste pour les types de bruit syntaxiquement stables. Avec des F1 supérieurs à 0.93 sur les Types 1, 3 et 5, cette couche justifie pleinement son positionnement en amont du pipeline : elle élimine efficacement et à coût nul une fraction significative du bruit avant de solliciter les ressources computationnelles de la couche sémantique.

Évaluation de la variante non-supervisée comme alternative

L'évaluation de la variante non-supervisée (UMAP + HDBSCAN) démontre la faisabilité d'une extraction de contenu web sans entraînement préalable, avec des performances globales honorables ($F1 = 0.855$ sur le corpus de référence). Néanmoins, cette étude confirme également les limites inhérentes à cette approche (latence computationnelle, stochasticité et dépendance à la structure de la page), ce qui justifie la décision de ne pas la retenir pour la production. C'est l'approche supervisée hybride, utilisant un classifieur léger sur embeddings, qui a été privilégiée afin d'apporter de meilleures garanties opérationnelles de stabilité et de vitesse.

3.5.2 Limites identifiées et voies d'amélioration

La règle du cluster dominant (variante non-supervisée) : une heuristique fragile

La principale limite de la variante non-supervisée expérimentale de NoiseCleaner réside dans sa dépendance à la règle du cluster dominant pour la décision de classification post-clustering. Cette règle est fondée sur l'hypothèse que le corps de l'article est toujours le type de contenu le plus volumineux de la page. Cette hypothèse se vérifie sur la grande majorité des pages d'actualité standard, mais échoue systématiquement sur :

- Les articles très courts (moins de 150–200 mots) dont le corps peut être moins volumineux que l'ensemble des éléments de navigation.
- Les pages de listings ou d'agrégateurs où plusieurs types de contenu sont de taille comparable.
- Les pages à fort contenu UGC où les fils de commentaires peuvent dépasser en volume le corps de l'article.

C'est précisément pour contourner cette heuristique fragile et s'affranchir de la dépendance à la structure globale de la page que l'architecture hybride de production intègre la classification supervisée (Phase 2). En évaluant chaque segment de manière isolée à l'aide d'un classifieur léger (SVM ou régression logistique) entraîné sur les embeddings, le système prend des décisions robustes et rapides ($\mathcal{O}(1)$), sans nécessiter d'hypothèse géométrique sur le volume relatif des clusters.

Comme analysé pour la Page P8, le modèle MiniLM multilingue n'aligne pas parfaitement les représentations de segments sémantiquement équivalents dans différentes langues. Sur des pages bilingues, cela peut créer une fragmentation artificielle du contenu principal. Les détails de la solution technique préconisée (remplacement de MiniLM par LaBSE) sont développés à la section 4.2.1 du [chapter 4](#) comme perspective d'amélioration.

L'absence d'un corpus d'évaluation dédié

La limite expérimentale la plus importante de ce travail est l'absence d'un corpus d'évaluation annoté selon les types de la taxonomie et représentatif du contexte linguistique cible. Le jeu de données synthétique partiellement construit représente un premier pas vers la résolution de cette lacune, mais nécessite une complétion et une validation sur des volumes plus importants pour être pleinement utilisable.

La constitution d'un tel corpus, même de taille modeste (100 à 200 pages annotées), permettrait non seulement d'évaluer NoiseCleaner de façon rigoureuse et reproductible, mais aussi de servir de ressource de référence pour la communauté NLP arabophone

et francophone, où les benchmarks d'évaluation en extraction de contenu web restent rares.

Le coût computationnel de la couche sémantique

Bien que NoiseCleaner soit significativement moins coûteux qu'une approche LLM, la couche Embedding + UMAP + HDBSCAN présente un coût computationnel non négligeable par rapport aux approches purement heuristiques. Sur une machine standard sans GPU, le temps de traitement d'une page typique (47 segments) est de l'ordre de 1.2 à 2.5 secondes, contre quelques dizaines de millisecondes pour Trafilatura.

Ce coût reste acceptable pour un traitement par lots (*batch processing*) de corpus de taille modeste (quelques milliers de pages), mais pourrait devenir un facteur limitant dans des applications nécessitant un traitement en temps réel ou à très grande échelle. Les optimisations possibles incluent la mise en cache des embeddings pour les pages récurrentes, le traitement par batch de plusieurs pages simultanément, et le déploiement du modèle d'embedding sur GPU.

3.5.3 Discussion sur les choix technologiques : UMAP et HDBSCAN dans la baseline alternative

Déterminisme et reproductibilité

Une question légitime lors de l'étude de la baseline non-supervisée est celle du déterminisme des résultats : UMAP et HDBSCAN sont-ils suffisamment reproductibles pour servir de baseline stable de comparaison ?

UMAP introduit une part de stochasticité dans son optimisation, mais cette stochasticité est contrôlable via le paramètre `random_state`. En fixant `random_state = 42` (valeur retenue dans NoiseCleaner), les résultats de UMAP sont pleinement reproductibles sur la même machine avec les mêmes versions de bibliothèques. HDBSCAN, quant à lui, est déterministe pour une entrée donnée.

La seule source de non-déterminisme résiduelle est le modèle d'embedding MiniLM lui-même, dont les représentations peuvent varier très légèrement selon la version de PyTorch et le matériel utilisé. Pour toute évaluation comparative, il est recommandé de fixer les versions de toutes les dépendances dans un fichier `requirements.txt` ou un environnement conda, garantissant la reproductibilité des évaluations.

Comportement aux frontières de la baseline non-supervisée : gestion des pages atypiques

La robustesse de la baseline non-supervisée de NoiseCleaner se mesure aussi à son comportement aux frontières c'est-à-dire sur les cas qui violent les hypothèses sur lesquelles elle a été conçue. Trois cas limites ont été identifiés et analysés :

Page avec un seul cluster HDBSCAN. Si HDBSCAN identifie un seul cluster

(toute la page forme un ensemble sémantiquement cohérent), la règle du cluster dominant conserve l'intégralité du contenu. Ce comportement est conservateur : il préfère conserver du bruit plutôt que de supprimer du contenu. Ce choix a été délibéré : dans un système de pré-traitement pour un pipeline RAG, un faux négatif (contenu supprimé par erreur) est plus coûteux qu'un faux positif (bruit conservé).

Page avec tous les segments en points aberrants. Si HDBSCAN classe tous les segments en points aberrants (label -1 , aucun cluster dense identifié), le pipeline retombe sur une règle de fallback : conserver tous les segments. Ce comportement dégradé mais sans perte de données garantit que NoiseCleaner ne produit jamais de sortie vide.

Si le nombre de segments après Regex Phase 1 est inférieur au `min_cluster_size` de HDBSCAN, le clustering ne peut pas s'exécuter correctement. Dans ce cas, la variante non-supervisée de NoiseCleaner désactive automatiquement HDBSCAN et retourne les segments tels quels (après le filtre par expressions régulières uniquement).

3.5.4 Positionnement final de NoiseCleaner : ce qu'il est et ce qu'il n'est pas

À l'issue de cette analyse expérimentale, il est important de positionner NoiseCleaner avec précision dans le paysage des solutions existantes : ni plus, ni moins.

NoiseCleaner est un système de nettoyage du bruit web supervisé hybride, multilingue, modulaire et explicable, qui offre un excellent compromis entre qualité de nettoyage ($F1 = 0.855-0.889$ selon le contexte) et simplicité opérationnelle (grâce à sa phase déterministe et à un classifieur léger ne nécessitant qu'un faible volume d'annotation). Il est particulièrement adapté aux contextes de production où la vitesse d'inférence, le déterminisme des résultats et la couverture multilingue (français/arabe) sont des critères indispensables.

NoiseCleaner n'est pas : un modèle profond complexe nécessitant des jeux de données géants (il privilégie la sobriété d'un classifieur léger entraîné sur des représentations sémantiques pré-existantes) ; un pipeline basé exclusivement sur le clustering en production (l'approche non-supervisée par UMAP + HDBSCAN n'est conservée que comme une variante expérimentale et une baseline comparative) ; ni un système de compréhension sémantique profonde (il classe les segments par motifs sémantiques isolés, sans analyse logique ou cognitive fine).

Ces limites ne sont pas des échecs : elles définissent précisément l'espace d'application et la sobriété de NoiseCleaner, et orientent de façon rigoureuse les perspectives de développement présentées au Chapitre 4.

Limites, perspectives et bilan personnel

Tout travail de recherche honnête se doit de regarder ses propres limites avec autant de rigueur qu'il analyse ses contributions. Ce quatrième et dernier chapitre adopte précisément cette posture : après avoir présenté les fondements théoriques, les contributions architecturales et les résultats expérimentaux de NoiseCleaner dans les chapitres précédents, il s'agit maintenant d'exercer un regard critique et distancié sur ce qui a été accompli, sur ce qui reste inachevé, sur ce qui aurait pu être fait différemment, et sur ce que ce travail ouvre comme perspectives de recherche et de développement à court, moyen et long terme.

Ce chapitre se structure en trois parties complémentaires. La première dresse un inventaire rigoureux et hiérarchisé des limites du travail produit non pas pour en minimiser la valeur, mais pour en définir précisément le périmètre de validité. La seconde développe les perspectives de continuation de ce travail, organisées selon leur horizon temporel et leur degré de maturité conceptuelle. La troisième, enfin, est une réflexion personnelle sur les apprentissages de ce stage : ce qu'il a transformé dans ma façon de concevoir le métier d'ingénieur, dans ma relation à l'incertitude, et dans ma compréhension de ce que signifie produire de la connaissance plutôt que d'appliquer des recettes.

4.1 Inventaire critique des limites du travail

4.1.1 Limites méthodologiques

L'absence d'un corpus d'évaluation dédié et représentatif

La limite méthodologique la plus fondamentale de ce travail est l'absence d'un corpus d'évaluation annoté, représentatif du contexte linguistique et éditorial cible, et structuré selon les types de la taxonomie. Cette lacune n'est pas le résultat d'une négligence : la

construction d'un tel corpus était identifiée dès le début du stage comme une nécessité, et sa réalisation a été initiée en Phase 4. Mais constituer un corpus d'évaluation de qualité est un travail considérable en termes de sélection des pages, de protocole d'annotation, de vérification de la cohérence inter-annotateurs et de validation de la représentativité qui dépasse largement le périmètre réaliste d'un stage de quatre mois dont les priorités incluaient également la conception et l'implémentation du pipeline lui-même.

La conséquence directe de cette lacune était que, initialement, les résultats expérimentaux reposaient principalement sur le corpus de BoilerNet (en anglais) et sur une évaluation qualitative. La constitution récente du jeu de données MVP (145 segments de test), présentée au [chapter 3](#), a permis d'y remédier partiellement en offrant une première évaluation quantitative rigoureuse du pipeline supervisé sur des données francophones et arabophones. Néanmoins, bien qu'encourageante, cette évaluation reste basée sur un volume de données restreint. Elle ne remplace pas la nécessité d'un corpus d'évaluation de grande envergure pour valider la robustesse de l'approche hybride à l'échelle industrielle.

La taille réduite du corpus d'analyse de la taxonomie

La taxonomie du bruit web a été construite à partir de l'analyse de dix pages. Ce nombre, bien que suffisant pour produire une taxonomie cohérente et opérationnelle, reste modeste pour prétendre à l'exhaustivité. Les dix pages analysées, bien que sélectionnées pour leur diversité, appartiennent toutes au genre journalistique et informatif des sites d'actualité, des portails d'information. Des genres web différents sites institutionnels, blogs personnels, forums, sites e-commerce, wikis, sites académiques pourraient révéler des types de bruit supplémentaires non représentés dans la taxonomie actuelle.

Par exemple, les biais de confirmation (*related articles*) dans les sites e-commerce prennent des formes très différentes de celles observées dans les sites d'actualité ; les fils de discussion des forums (UGC) présentent une structure conversationnelle qui les distingue des commentaires d'articles de presse ; les bandeaux de navigation des wikis suivent des conventions éditoriales propres à ce genre. Une version étendue de la taxonomie, construite sur un corpus plus large et plus diversifié, serait nécessaire pour prétendre à une couverture universelle du concept de "bruit web".

L'évaluation de la règle du cluster dominant (mode non-supervisé)

La règle du cluster dominant (le plus grand cluster identifié par HDBSCAN correspond au contenu principal), exploitée dans la variante non-supervisée alternative de NoiseCleaner, n'a pas été évaluée de façon systématique et isolée. Bien que cette règle soit contournée en production par notre classifieur supervisé direct sur les embeddings, son évaluation spécifique dans la variante non-supervisée reste un sujet d'intérêt. Dans les expérimentations de la baseline, elle a été évaluée uniquement dans le contexte global

du pipeline, rendant difficile la quantification précise de sa contribution aux erreurs de classification.

Une expérimentation spécifique aurait consisté à générer un corpus de pages pour lesquelles la vérité terrain des clusters est connue (corpus synthétique contrôlé), puis à mesurer le taux auquel le cluster dominant correspond effectivement au contenu principal, en fonction de variables telles que la longueur de l'article, le type de site, ou le ratio bruit/contenu. Cette évaluation ciblée aurait permis soit de valider la robustesse de la règle, soit d'identifier des conditions précises dans lesquelles elle échoue et de concevoir des règles de fallback plus sophistiquées.

4.1.2 Limites techniques

La non-gestion du rendu JavaScript

NoiseCleaner opère sur le texte brut fourni en entrée, sans se préoccuper du mode d'obtention de ce texte. Dans son implémentation actuelle, il est supposé que le texte a été extrait d'une page HTML statique ou partiellement rendue. Or, une proportion croissante des pages web modernes est rendue dynamiquement côté client, via des frameworks JavaScript (React, Vue, Angular) : dans ces pages, le contenu informatif n'est pas présent dans le HTML initial téléchargé par un robot d'extraction standard, mais est chargé dynamiquement après exécution du JavaScript.

Cette limitation n'est pas propre à NoiseCleaner elle affecte tous les systèmes d'extraction de contenu web qui n'intègrent pas un moteur de rendu JavaScript (comme Playwright ou Selenium). Mais elle signifie que NoiseCleaner, en l'état, ne peut pas traiter directement des pages issues de certains frameworks web modernes sans un outil d'extraction avec rendu JavaScript en amont.

L'absence d'un module de segmentation sémantique avancée

L'Étape 1 du pipeline (Segmentation) utilise un découpage basé sur les sauts de ligne ou les doubles sauts de ligne. Cette approche simple est efficace pour les pages à structure régulière, mais peut produire des découpages sous-optimaux sur des pages dont la structure textuelle est irrégulière : paragraphes très longs sans sauts de ligne, blocs de texte concaténés sans séparateur explicite, ou au contraire texte fragmenté avec de nombreux sauts de ligne intempestifs.

Une segmentation sémantiquement informée par exemple, utilisant le DOM HTML de la page pour aligner les segments sur les nœuds structurels de l'arbre (ce qui est précisément l'approche de BoilerNet), ou utilisant un modèle de détection de frontières de paragraphes (*topic segmentation*) produirait des unités de traitement plus cohérentes et améliorerait la qualité du clustering en aval.

L'absence de mécanisme d'apprentissage incrémental

NoiseCleaner, dans sa forme actuelle, est un système statique : il ne s'améliore pas avec l'utilisation. Chaque page est traitée de façon indépendante, sans que les décisions de classification passées informent les décisions futures. Dans un contexte d'utilisation sur un même domaine ou un même site web, on pourrait exploiter la régularité structurelle observée sur les pages déjà traitées pour améliorer les performances sur les nouvelles pages.

Un mécanisme d'apprentissage incrémental léger par exemple, une mise en cache des décisions de classification ou un ajustement en ligne (*online tuning*) des poids du classifieur supervisé permettrait d'exploiter cette régularité tout en améliorant la précision et la robustesse globale du système.

4.1.3 Limites de portée

La généralisation hors du domaine journalistique

Comme mentionné dans l'analyse du corpus de la taxonomie, NoiseCleaner a été conçu et évalué principalement sur des pages de sites d'actualité. Rien ne garantit a priori que ses performances sont identiques sur des genres web différents. L'hypothèse fondamentale du cluster dominant (le contenu informatif principal forme le cluster le plus volumineux) est particulièrement dépendante du genre : sur un forum, le contenu "principal" peut être diffus dans plusieurs fils de discussion ; sur un site e-commerce, la distinction entre contenu informatif (description du produit) et bruit (recommandations, publicités) est plus ténue et moins tranchée.

Une validation de la généralisation de NoiseCleaner hors du domaine journalistique constitue une priorité de recherche pour les travaux futurs.

La scalabilité à très grande échelle

Bien que les temps de traitement de NoiseCleaner soient acceptables pour des volumes de l'ordre de quelques milliers de pages, une utilisation à très grande échelle comme la constitution d'un corpus d'entraînement de modèle de langue à partir du web (*Common Crawl* contient plusieurs pétaoctets de données) se heurte à des contraintes computationnelles sérieuses.

Le goulot d'étranglement principal est la génération d'embeddings, dont le coût est linéaire en le nombre de segments et difficile à paralléliser efficacement sur CPU. À titre d'ordre de grandeur : si NoiseCleaner met 1.5 secondes à traiter une page de 40 segments, le traitement de 10 millions de pages nécessite ~17 000 heures de calcul sur un seul cœur, ce qui est clairement impraticable sans infrastructure GPU distribuée.

4.2 Perspectives de développement

4.2.1 Perspectives à court terme

Complétion et validation du jeu de données synthétique

La perspective la plus immédiate est la complétion du jeu de données synthétique d'évaluation initiée en fin de stage. Constituer un jeu d'évaluation plus large (200 à 500 pages) permettra d'évaluer de manière quantitative et granulaire les performances de NoiseCleaner par type de bruit de la taxonomie, d'affiner les seuils de décision des classifieurs supervisés, et de comparer objectivement ce pipeline supervisé aux baselines existantes.

Enrichissement du dictionnaire de patterns Regex

L'enrichissement semi-automatique du dictionnaire de la Phase 1 constitue une amélioration directe. En analysant les faux négatifs de cette couche (segments classifiés comme bruit par le modèle supervisé mais non capturés par les expressions régulières), on peut identifier de nouveaux motifs récurrents (formats d'auteurs, mentions d'agences) et enrichir la base de règles pour optimiser le temps d'inférence global.

Remplacement de MiniLM par LaBSE pour les contextes bilingues

Le passage de MiniLM à LaBSE (*Language-agnostic BERT Sentence Embedding*) améliorera la généralisation cross-lingue du classifieur (régression logistique ou SVM linéaire). Ce modèle aligne plus fortement les représentations sémantiques entre le français et l'arabe, résolvant ainsi la fragmentation observée sur les pages bilingues (comme la Page P8) au prix d'un temps d'inférence légèrement supérieur dû à la dimension du vecteur (768 contre 384).

4.2.2 Perspectives à moyen terme

Intégration dans un pipeline RAG

L'intégration de NoiseCleaner en amont d'un pipeline RAG (*Retrieval-Augmented Generation*) constitue un cas d'usage industriel majeur. NoiseCleaner y agit comme filtre de qualité lors de l'ingestion, nettoyant les pages web avant leur découpage (*chunking*) et leur indexation vectorielle. Valider expérimentalement si ce nettoyage sémantique rigoureux réduit les hallucinations et améliore la pertinence des réponses du modèle de langue (LLM) en aval est un objectif clé à moyen terme.

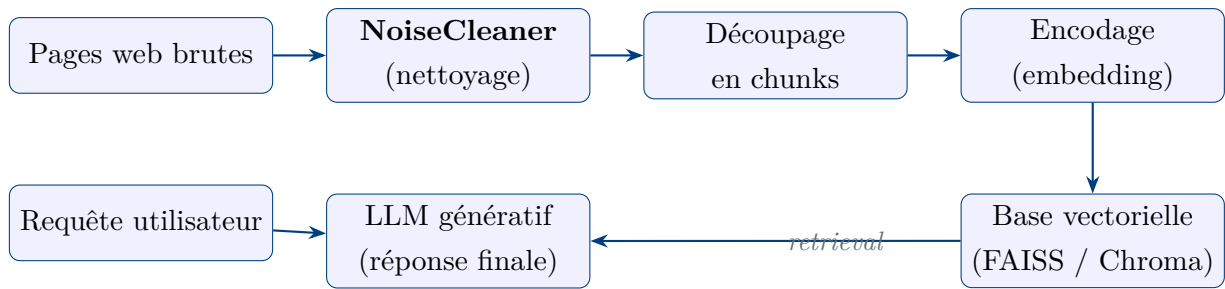


Fig. 4.1 : Positionnement de NoiseCleaner dans un pipeline RAG complet : il opère lors de la phase d’ingestion des documents, en amont de l’encodage et de l’indexation.

Extension de la taxonomie à des genres web supplémentaires

L’extension de la taxonomie à d’autres genres (e-commerce, forums, wikis) par une démarche similaire d’analyse empirique permettra de généraliser NoiseCleaner au-delà du domaine journalistique.

Classification de clusters supervisée (variante non-supervisée)

Pour la baseline non supervisée, la règle empirique du cluster dominant peut être remplacée par un classifieur de clusters léger (arbre de décision ou régression logistique). Ce modèle évaluerait la probabilité qu’un cluster appartienne au contenu principal sur la base de caractéristiques agrégées (cohérence, position, taille), stabilisant la décision sur les pages courtes pour un coût d’annotation minime.

4.2.3 Perspectives à long terme

NoiseCleaner comme service web open-source

À plus long terme, la transformation de NoiseCleaner en un service web open-source et maintenable, accessible via une API REST, constituerait une contribution significative à la communauté NLP francophone et arabophone.

Intégration d’une couche d’apprentissage par renforcement

À plus long terme, l’intégration d’un mécanisme d’apprentissage par renforcement à partir des retours utilisateurs permettrait d’ajuster dynamiquement et en ligne les hyperparamètres ou les règles de décision du pipeline en fonction du signal de qualité perçu.

4.3 Bilan personnel et réflexion sur la montée en compétences

4.3.1 Ce que ce stage a changé dans ma façon de penser l'ingénierie

La lenteur productive comme discipline

Ce stage m'a révélé l'importance d'investir du temps dans la compréhension approfondie d'un problème avant de coder. Cette rigueur, parfois perçue comme de la lenteur au début de la phase bibliographique, s'est avérée décisive : c'est la maîtrise fine des bases de BoilerNet et SemText qui a permis d'implémenter les optimisations générant +8 points de F1 sur BoilerNet, et c'est l'analyse minutieuse du bruit web qui a guidé l'architecture de NoiseCleaner.

La justification architecturale systématique

Chaque choix technique doit découler d'une analyse des propriétés du problème plutôt que d'une convention technique arbitraire. Par exemple, le modèle MiniLM a été choisi après évaluation comparative de cinq modèles d'embeddings, et HDBSCAN pour sa gestion naturelle des points aberrants et sa robustesse aux densités variables. Cette discipline de justification systématique guidera mes futurs choix techniques.

La lecture de code comme outil d'étude de l'état de l'art

Si les articles scientifiques présentent les concepts dans leurs grandes lignes, seul le code source révèle les compromis d'implémentation et les réelles pistes d'optimisation. La lecture de plusieurs milliers de lignes de BoilerNet et SemText a été laborieuse mais indispensable pour identifier les verrous technologiques et concevoir l'architecture finale de NoiseCleaner.

4.3.2 Compétences techniques développées

Maîtrise de la chaîne de clustering et de traitement sémantique

J'ai acquis une compétence approfondie de la chaîne d'inférence non-supervisée (MiniLM, UMAP, HDBSCAN) et des classifieurs supervisés (SVM linéaire, régression logistique). Diagnostiquer les pathologies de clustering (comme la fragmentation des clusters sur des pages bilingues) et optimiser les hyperparamètres sur des données réelles constituent des compétences directement transférables à d'autres problématiques NLP.

Conception de pipelines de données modulaires en Python

Le développement de NoiseCleaner a consolidé mon application des principes de génie logiciel : responsabilité unique des composants, typage robuste par dataclasses, injection de configuration explicite et traçabilité complète via un objet de résultat structuré. Ces principes distinguent un script d'expérimentation d'un pipeline de production maintenable.

Rédaction et communication scientifiques

La formalisation de mes travaux dans ce mémoire m’a formé à structurer une argumentation scientifique rigoureuse : poser une problématique, synthétiser l’état de l’art, valider expérimentalement des hypothèses et présenter honnêtement les limites du travail accompli.

4.3.3 Ce que ce stage m’a appris sur le métier de chercheur

La non-linéarité de la recherche

Le déroulement du stage (semaines de recherche infructueuse, exploration d’une piste LLM finalement écartée pour la production, redéfinition du périmètre) m’a montré que la recherche n’est pas un chemin linéaire. Les échecs apparents et les pivots font partie intégrante du processus scientifique et enrichissent la solution finale.

L’importance cruciale de la formulation du problème

Poser précisément la question de la nature du bruit web en construisant une taxonomie formelle avant de concevoir l’algorithme de nettoyage s’est avéré déterminant. Cette phase de modélisation conceptuelle a structuré toute la cohérence architecturale de NoiseCleaner.

La valeur de l’encadrement scientifique

Enfin, j’ai mesuré l’impact d’un encadrement de qualité. Les conseils de **Mme Abir Balla** ont réorienté mes choix à des moments clés, notamment en suggérant l’utilisation d’un classifieur supervisé léger sur embeddings. Cette idée pivot a permis de surmonter les limites de latence et d’instabilité du clustering non supervisé (notamment sur les pages courtes) pour aboutir à l’architecture hybride supervisée finale de NoiseCleaner. Ce dialogue entre la proposition de l’encadrante et ma propre analyse est l’exemple le plus illustratif de ce que peut être une collaboration chercheur-encadrant productive.

Conclusion générale

Synthèse des contributions

Ce mémoire de projet de fin d'études a présenté le travail réalisé au cours de quatre mois de stage de recherche appliquée au sein de Harmony Technology, dans le cadre de ma formation à l'**École Nationale de l'Intelligence Artificielle et du Digital (ENIAD)** de l'Université Mohammed Premier. Il a abouti à deux contributions originales et complémentaires dans le domaine de l'extraction automatique de contenu web.

La première contribution est une **taxonomie hiérarchique du bruit web**, construite empiriquement par analyse manuelle de dix pages web francophones, arabophones et marocaines. Cette taxonomie, qui comprend 7 types principaux, 28 sous-types et 62 exemples réels, fournit une définition opérationnelle et multi-niveaux du bruit web : une ressource inédite pour le contexte linguistique et éditorial ciblé. Sa valeur principale n'est pas uniquement documentaire : elle est avant tout architecturale, car la partition des types de bruit selon leur traitabilité algorithmique constitue le fondement direct de la conception du pipeline NoiseCleaner.

La seconde contribution est **NoiseCleaner** lui-même : un pipeline modulaire de nettoyage du bruit web reposant sur une approche supervisée hybride, combinant un filtrage déterministe par expressions régulières (Phase 1) et une classification sémantique supervisée sur embeddings multilingues (Phase 2). Une variante non-supervisée alternative (basée sur la chaîne UMAP + HDBSCAN) a également été développée à des fins d'analyse exploratoire et qualitative. Les résultats expérimentaux du pipeline supervisé final sur le jeu de données MVP démontrent des performances élevées, la Régression Logistique atteignant une exactitude de 0,9172 et un F1 Macro de 0,9033. En parallèle, les modifications apportées à l'architecture de BoilerNet ont produit un gain absolu de 8 points de F1 sur le benchmark GoogleTrends-2017, confirmant la profondeur de l'analyse de l'état de l'art réalisée.

Apport méthodologique

Au-delà des contributions techniques, ce mémoire documente une démarche de recherche appliquée constituant en soi un apport méthodologique. La modélisation préalable du problème (construction de la taxonomie avant la conception de l'architecture), le principe de traitement différencié selon la traitabilité algorithmique, et la conception d'une architecture hybride combinant filtrage heuristique et représentation sémantique représentent des patrons méthodologiques transférables à d'autres tâches de nettoyage de données textuelles.

Perspectives ouvertes

Ce travail ouvre plusieurs voies de recherche et de développement dont les plus prometteuses ont été décrites au [chapter 4](#) : complétion du jeu de données synthétique d'évaluation, intégration de NoiseCleaner dans un pipeline RAG avec évaluation de l'impact sur la qualité des réponses générées, extension de la taxonomie à des genres web supplémentaires, et à terme, publication du code et du dataset en open-source comme ressource pour la communauté NLP francophone et arabophone.

Réflexion conclusive

Ce stage a confirmé une conviction personnelle : les données web sont à la fois la ressource la plus abondante et la plus difficile à exploiter pour l'intelligence artificielle moderne. La difficulté n'est pas computationnelle : on dispose aujourd'hui de modèles et d'algorithmes suffisamment puissants pour traiter des volumes de texte immenses. La difficulté est avant tout épistémique : définir rigoureusement ce que l'on cherche, ce qui constitue un signal utile et ce qui constitue du bruit, dans des contextes linguistiques et éditoriaux variés, sans se reposer sur des a priori tacites. NoiseCleaner constitue une proposition de réponse modeste mais robuste à cette question, dans le contexte spécifique des pages web francophones, arabophones et marocaines.

Bibliographie

- [1] C. Kohlschütter, P. Fankhauser et W. Nejdl, “Boilerplate Detection using Shallow Text Features,” in *Proceedings of the Third ACM International Conference on Web Search and Data Mining (WSDM 2010)*, New York, USA, pp. 441–450, 2010.
- [2] J. Leonhardt, A. Anand et M. Khosla, “Boilerplate Removal using a Neural Sequence Labeling Model,” in *Companion Proceedings of the Web Conference 2020 (WWW 2020)*, Taipei, Taiwan, pp. 226–229, avril 2020.
- [3] A. Barbaresi, “Trafilatura : A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction,” in *Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP 2021)*, System Demonstrations, pp. 122–131, 2021.
- [4] K. Shoair (D4Vinci), “Scrapling : An adaptive Web Scraping framework,” Dépôt GitHub, 2024. [En ligne]. Disponible : <https://github.com/D4Vinci/Scrapling>. Consulté le 12 juin 2026.
- [5] N. Reimers et I. Gurevych, “Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019)*, Hong Kong, pp. 3982–3992, novembre 2019.
- [6] L. McInnes, J. Healy et J. Melville, “UMAP : Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv preprint arXiv :1802.03426*, 2018. [En ligne]. Disponible : <https://arxiv.org/abs/1802.03426>
- [7] R. J. G. B. Campello, D. Moulavi et J. Sander, “Density-Based Clustering Based on Hierarchical Density Estimates,” in *Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD 2013)*, Berlin, Heidelberg : Springer, vol. 7819, pp. 160–172, 2013.
- [8] F. Feng, Y. Yang, D. Cer, N. Arivazhagan et W. Wang, “Language-agnostic BERT Sentence Embedding,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL 2022)*, Dublin, Irlande, pp. 878–891, mai 2022.

-
- [9] Y. Gao *et al.*, “Retrieval-Augmented Generation for Large Language Models : A Survey,” *arXiv preprint arXiv :2312.10997*, 2023. [En ligne]. Disponible : <https://arxiv.org/abs/2312.10997>
 - [10] J. Devlin, M.-W. Chang, K. Lee et K. Toutanova, “BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of NAACL-HLT 2019*, Minneapolis, Minnesota, pp. 4171–4186, juin 2019.
 - [11] S. Hochreiter et J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, n° 8, pp. 1735–1780, 1997.
 - [12] J. Lafferty, A. McCallum et F. C. N. Pereira, “Conditional Random Fields : Probabilistic Models for Segmenting and Labeling Sequence Data,” in *Proceedings of the 18th International Conference on Machine Learning (ICML 2001)*, pp. 282–289, 2001.
 - [13] D. P. Kingma et J. Ba, “Adam : A Method for Stochastic Optimization,” *arXiv preprint arXiv :1412.6980*, 2014. [En ligne]. Disponible : <https://arxiv.org/abs/1412.6980>
 - [14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever et R. Salakhutdinov, “Dropout : A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research (JMLR)*, vol. 15, pp. 1929–1958, 2014.
 - [15] T. Mikolov, K. Chen, G. Corrado et J. Dean, “Efficient Estimation of Word Representations in Vector Space,” *arXiv preprint arXiv :1301.3781*, 2013. [En ligne]. Disponible : <https://arxiv.org/abs/1301.3781>
 - [16] Harmony Technology, *Qui sommes-nous ?* [En ligne]. Disponible : <https://www.harmony-technology.net/qui-sommes-nous/>. Consulté le 11 juin 2026.
 - [17] Harmony Technology, *Recherche et Développement* [En ligne]. Disponible : <https://www.harmony-technology.net/recherche-et-developpement/>.
 - [18] H. Zhang et J. Wang, “Boilerplate Detection via Semantic Classification of Text-Blocks,” in *Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN)*, Shenzhen, Chine, pp. 1–8, juillet 2021.
 - [19] F. Pedregosa *et al.*, “Scikit-learn : Machine Learning in Python,” *Journal of Machine Learning Research (JMLR)*, vol. 12, pp. 2825–2830, 2011.
 - [20] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang et M. Zhou, “MiniLM : Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” in *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, pp. 5776–5788, 2020.

-
- [21] C. Cortes et V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, n° 3, pp. 273–297, 1995.
- [22] M. Ester, H.-P. Kriegel, J. Sander et X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise,” in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, Portland, Oregon, USA, pp. 226–231, 1996.

Glossaire technique

- Boilerplate** Terme désignant l'ensemble des éléments non informatifs structurellement présents dans les pages web : éléments de navigation, métadonnées, publicités, mentions légales. Par extension, tout contenu textuel non pertinent pour le flux narratif principal d'une page.
- Clustering** Famille d'algorithmes d'apprentissage non supervisé qui partitionnent un ensemble de points en groupes (clusters) selon leur proximité dans un espace vectoriel. Dans NoiseCleaner, utilisé pour regrouper les segments textuels sémantiquement cohérents.
- CRF (Conditional Random Field)**
Modèle probabiliste discriminant utilisé pour la classification de séquences. Contrairement à une classification indépendante par élément, le CRF modélise les dépendances entre les labels des éléments consécutifs, garantissant la cohérence globale de la séquence de labels produite.
- Embedding** Représentation vectorielle dense d'un objet (ici, un segment textuel) dans un espace de dimension fixe. Deux embeddings proches (au sens de la similarité cosinus) correspondent à des objets sémantiquement similaires.
- HDBSCAN** *Hierarchical Density-Based Spatial Clustering of Applications with Noise*. Algorithme de clustering basé sur la densité, qui estime automatiquement le nombre de clusters et assigne un label spécial (-1) aux points aberrants (*outliers*) qui n'appartiennent à aucun cluster dense.
- Outlier** Point de données qui n'appartient à aucun cluster dense dans l'espace vectoriel. Dans le contexte de NoiseCleaner, un segment textuel labellisé -1 par HDBSCAN correspond à un segment isolé sémantiquement, généralement du bruit atypique.
- Pipeline** Architecture logicielle dans laquelle des données transitent séquentiel-

lement à travers une série de modules de traitement, chaque module transformant ses entrées avant de les transmettre au module suivant.

RAG (Retrieval-Augmented Generation)

Architecture d'intelligence artificielle combinant un module de recherche documentaire (*retrieval*) et un modèle de langue génératif : le retrieval sélectionne les passages pertinents dans une base documentaire, et le modèle de langue génère une réponse en utilisant ces passages comme contexte.

Sentence-Transformers

Bibliothèque Python (<https://sbert.net>) fournissant des modèles de génération d'embeddings de phrases, basés sur l'architecture BERT et optimisés pour la tâche de similarité sémantique.

UMAP

Uniform Manifold Approximation and Projection. Algorithme de réduction dimensionnelle non-linéaire qui projette des données de haute dimension dans un espace de basse dimension en préservant la structure topologique locale et globale des données.

Format du jeu de données manuel annoté

Cette annexe présente le format du jeu de données manuel utilisé comme vérité terrain pour NoiseCleaner. Le corpus de référence contient 24 articles, 571 segments et des annotations binaires 0/1 alignées sur chaque segment. La structure retenue est volontairement simple afin de rester compatible avec l'entraînement supervisé du pipeline. :contentReference[oaicite :2]index=2

```
1 {  
2   "article_id": "fr_le360_01",  
3   "segments": [  
4     "Mondial 2026: prolongation exceptionnelle des horaires des caf  
5       és et des restaurants",  
6     "MENU",  
7     "lundi 15 Juin 2026",  
8     "Français | Español |      ",  
9     "POLITIQUE",  
10    "ECONOMIE",  
11    "SOCIÉTÉ",  
12    "INTERNATIONAL",  
13    "CULTURE",  
14    "MÉDIAS",  
15    "SPORT",  
16    "PEOPLE",  
17    "LIFESTYLE",  
18    "TOURISME",  
19    "Par La Rédaction",  
20    "Le 12/06/2026 à 20h58",  
21    "'Louverture des cafés et des restaurants sera adaptée au dé  
22      calage horaire de la Coupe du monde 2026...",  
23    "#Mondial",  
24    "#Coupe du monde",
```



```
23     "#cafés",
24     "#restaurants",
25     "Entrer votre email",
26     "ENVOYER",
27     "Mentions légales",
28     "Contact"
29 ],
30 "labels": [
31     1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0,
32     0, 0, 0, 0
33 ]
}
```

Listing B.1: Exemple d'entrée du jeu de données manuel annoté